# Explanatory Data Analysis

Sofia Rivas and Jocelyn Rojas

## 1. Research Question and Background

To what extent do biological and environmental variables predict **sea urchin gonad index** across kelp forests and urchin barrens, and which variables are the strongest predictors of gonad index?

## 2. Load Relevant Packages

```
library(tidyverse) #data wrangling (includes other packages)
library(dplyr) #data wrangling
library(ggplot2) #plotting
library(brms) #Bayesian statistics
library(bayesplot) #posterior probability distributions
library(factoextra) #variance plot (scree plot)
library(tidybayes) #helps convert Bayesian modeling for ggplot2 and tidyverse
library(patchwork) #put plots together
```

## 3. Data Wrangling

The data for this project came from a Monterey Bay Aquarium kelp forest recovery research program. From 2024-2025, data was collected during the summer field seasons. Data collected included SCUBA monitoring surveys (recording species density, kelp cover and topography) and purple sea urchin collection from 12 sites around Carmel Bay and the Monterey Bay peninsula. Sea urchins were dissected in the lab to calculate mean gonadosomatic index for each zone within sites.

The raw dataset contains ~80 variables with 98 observations. To simplify the dataset for more accurate analysis, we selected 20 variables that we deemed as most relevant to our research question.

```
#raw data
predictors_raw <- readRDS("quad_build3.rds")

#select relevant columns and delete rows with NA values
predictors <- predictors_raw %>%
   mutate(juveniles = macj + nerj + ptej + lsetj + eisj,
         n_macro_plants_m2 = n_macro_plants_20m2/20,
         macro_stipe_density_m2 = macro_stipe_density_20m2/20,
         adults = (density20m2_ptecal + density20m2_eisarb + density20m2_nerlue + density20m2
         marine_snails = tegula_densitym2 + pomaulax_densitym2,
         density_cancer = density20m2_cancer_spp/20,
         density_lamstump = density20m2_lamstump/20,
         density_macstump = density20m2_macstump/20) %>%
  dplyr::select(mean_gi, n_macro_plants_m2, macro_stipe_density_m2, adults,
juveniles, density_cancer, density_lamstump, density_macstump, relief_cm, risk_index, purple_
  na.omit()

#final dataset for project
predictors2 <- predictors %>%
  dplyr::select(mean_gi, cov_mac_holdfast_live, macro_stipe_density_m2, n_macro_plants_m2, co
```

## 3. Principal Component Analysis

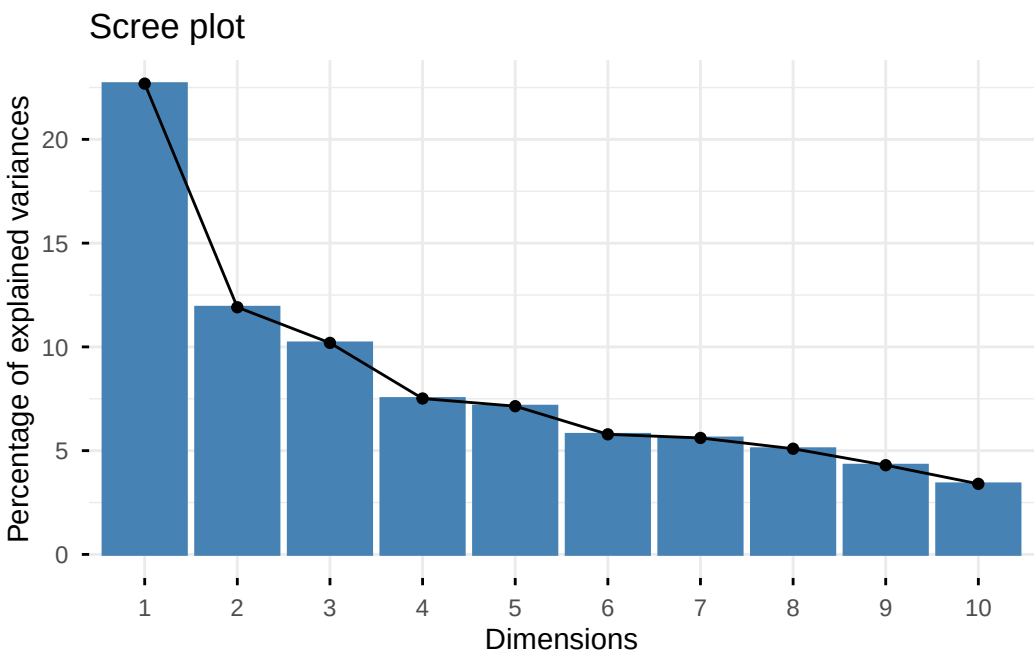
### Why we chose PCA

Even though we narrowed our dataset, we still had too many variables for the number of observations to produce a reliable model. To deal with this, we used Principal Component Analysis. PCA is a dimensionality reduction technique used to simplify complex datasets by transforming potentially correlated variables (e.g., kelp stipe density, holdfast cover, macroalgal abundance) into a smaller set of variables, called principal components.

### Step 1: PCA

```
#run PCA: run only on predictor variables, not the response variable (mean_gi)
#standardize variables
```

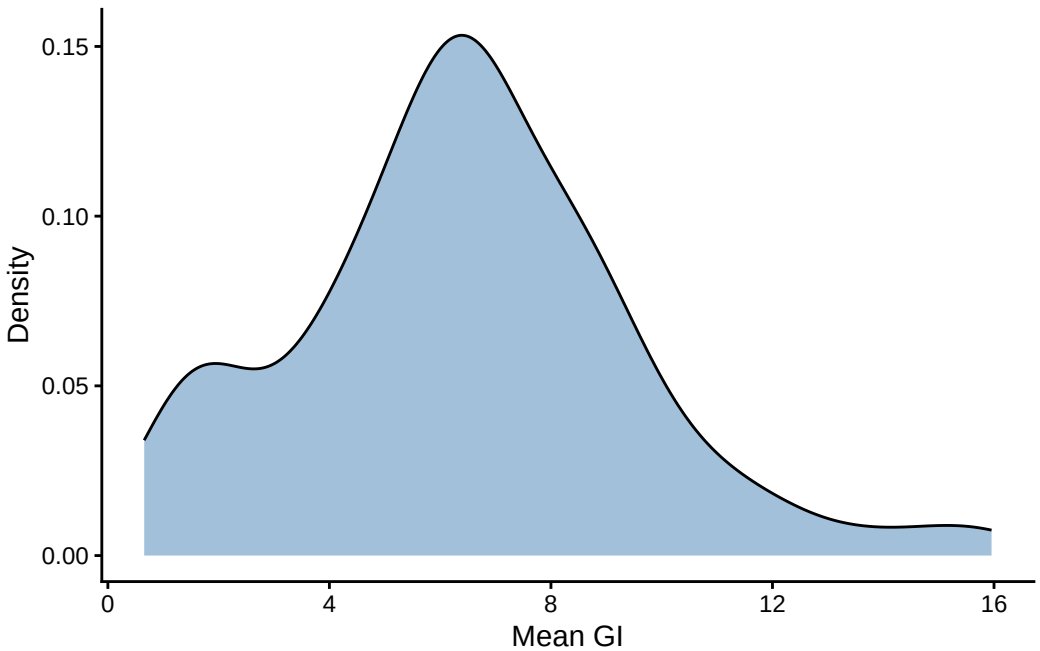
```r
pca <- prcomp(predictors2 %>%
                dplyr::select(-mean_gi),
                scale. = TRUE)

#visualize
fviz_eig(pca)
```



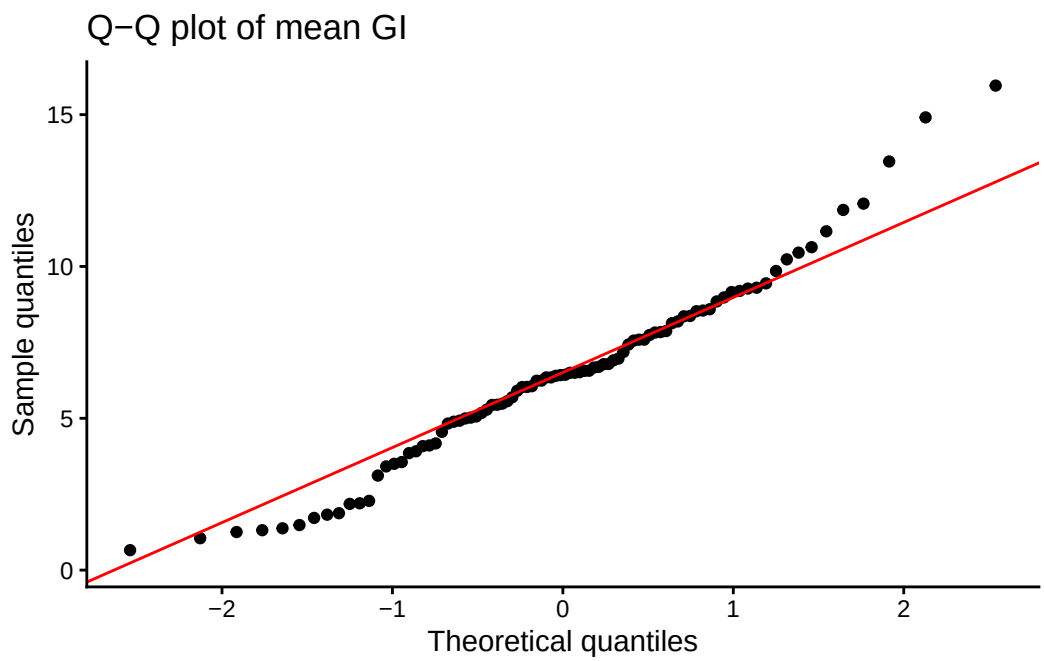

### Step 2: BRM Model Setup

```r
#create matrix of raw PC scores (reference scree plot)
pc_scores <- as.data.frame(pca$x[, 1:10])

#add response variable (mean_gi) into new dataframe
pc_data <- pc_scores %>%
              mutate(mean_gi = predictors2$mean_gi)

#set priors for brm using PCA
priors_pca <- c(
  prior(normal(0, 1), class = "b"),
  prior(normal(6.5, 2), class = "Intercept"), #mean mean_gi is ~6.5
```

```r
  prior(student_t(3, 0, 2), class = "sigma"))

#density plot to check for normality of the data
ggplot(pc_data, aes(x = mean_gi)) +
  geom_density(fill = "steelblue", alpha = 0.5) +
  labs(x = "Mean GI", y = "Density") +
  theme_classic()
```




```r
#Q-Q plot as another way to check for normality
ggplot(pc_data, aes(sample = mean_gi)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  labs(x = "Theoretical quantiles",
       y = "Sample quantiles",
       title = "Q-Q plot of mean GI") +
  theme_classic()
```




### Step 3: Model Selection

After checking the data's distribution and running the gaussian brm model, we wanted to see if there may be a better distribution family because the distribution is skewed. We then used the posterior predictive checks with pp_check() to visualize the model fit. Then, we re-ran the brm model with only the significant principal components (PCs) from the first gaussian model and used the same process for a skew-normal model. Comparing the model_pca_gaus with the model_pca_skew using loo_compare(), the skew-normal distribution is a better fit for predictive power.

```
#run brm model with default normal/gaussian distribution
model_pca <- brm(mean_gi ~ .,
                 data = pc_data,
                 family = gaussian(),
                 prior = priors_pca,
                 chains = 4, iter = 4000, warmup = 2000,
                 cores = 4)
```

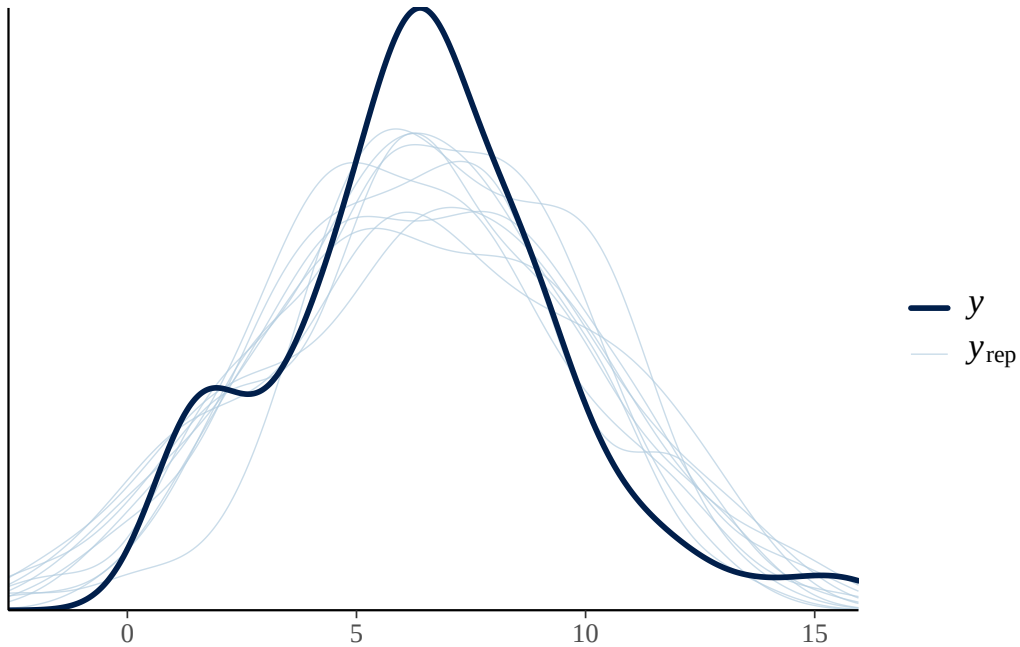
```
summary(model_pca) #PC1, PC2, and PC3 are significant!
```

```
 Family: gaussian
```

```
  Links: mu = identity
Formula: mean_gi ~ PC1 + PC2 + PC3 + PC4 + PC5 + PC6 + PC7 + PC8 + PC9 + PC10
   Data: pc_data (Number of observations: 90)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     6.44      0.25     5.95     6.92 1.00    18189     5587
PC1           0.82      0.12     0.58     1.06 1.00    16205     5982
PC2          -0.44      0.17    -0.78    -0.10 1.00    18267     5347
PC3           0.46      0.18     0.10     0.81 1.00    16076     5970
PC4           0.08      0.21    -0.33     0.50 1.00    17550     6050
PC5           0.04      0.22    -0.38     0.46 1.00    18145     5848
PC6           0.08      0.24    -0.39     0.57 1.00    18356     5522
PC7          -0.26      0.24    -0.74     0.23 1.00    16823     6003
PC8          -0.07      0.26    -0.58     0.43 1.00    17872     5245
PC9          -0.13      0.28    -0.67     0.41 1.00    19016     5997
PC10          0.17      0.31    -0.45     0.77 1.00    20538     5163

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     2.43      0.20     2.09     2.86 1.00    11401     5339

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
#posterior predictive check
pp_check(model_pca) #mostly fits well, doesn't account for bump around x=2
```

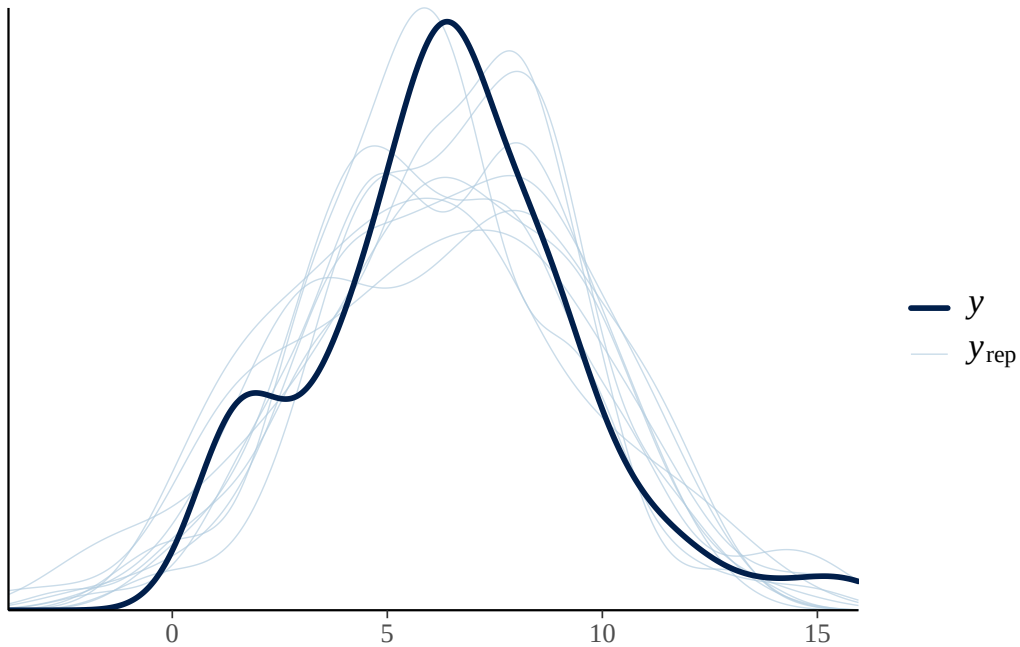
```
Using 10 posterior draws for ppc type 'dens_overlay' by default.
```




```
#re-run with only significant PCs
model_pca_gaus <- brm(mean_gi ~ PC1 + PC2 + PC3,
                                    data = pc_data,
                                    family = gaussian(),
                                    prior = priors_pca,
                                    chains = 4, iter = 4000, warmup = 2000,
                                    cores = 4)
```

```
pp_check(model_pca_gaus)
```

```
Using 10 posterior draws for ppc type 'dens_overlay' by default.
```




```r
#rerun first model with skew_normal distribution
model_pca_other <- brm(mean_gi ~ .,
                       data = pc_data,
                       family = skew_normal(), #changed to skew_normal
                       prior = priors_pca,
                       chains = 4, iter = 4000, warmup = 2000,
                       cores = 4)
```

```r
summary(model_pca_other) #again, PC1, PC2, and PC3 are significant!
```

```
 Family: skew_normal
  Links: mu = identity
Formula: mean_gi ~ PC1 + PC2 + PC3 + PC4 + PC5 + PC6 + PC7 + PC8 + PC9 + PC10
   Data: pc_data (Number of observations: 90)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     6.48      0.24     6.05     6.97 1.00     6619     5994
PC1           0.77      0.10     0.57     0.96 1.00     6478     5528
PC2          -0.57      0.13    -0.82    -0.31 1.00     8108     6130
```

```
PC3           0.38      0.14     0.12     0.68 1.00     6194     5221
PC4           0.05      0.15    -0.26     0.35 1.00     7873     5904
PC5           0.00      0.16    -0.31     0.33 1.00     8457     5956
PC6          -0.11      0.16    -0.41     0.21 1.00     7719     5826
PC7          -0.09      0.21    -0.50     0.32 1.00     8567     5928
PC8          -0.19      0.21    -0.61     0.23 1.00     7942     5775
PC9          -0.11      0.23    -0.56     0.34 1.00     7393     6485
PC10         -0.03      0.22    -0.45     0.42 1.00     7958     6058

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     2.32      0.19     1.98     2.72 1.00     5911     5609
alpha     5.76      1.95     2.62    10.16 1.00     5634     4469

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
#final model
model_pca_skew <- brm(mean_gi ~ PC1 + PC2 + PC3,  # only significant PCs
                    data = pc_data,
                    family = skew_normal(),
                    prior = priors_pca,
                    chains = 4, iter = 4000, warmup = 2000,
                    cores = 4)
```

```r
summary(model_pca_skew)
```

```
 Family: skew_normal
  Links: mu = identity
Formula: mean_gi ~ PC1 + PC2 + PC3
   Data: pc_data (Number of observations: 90)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     6.49      0.23     6.06     6.95 1.00     6233     5179
PC1           0.80      0.09     0.62     0.96 1.00     7486     5451
PC2          -0.58      0.12    -0.82    -0.34 1.00     8158     6207
PC3           0.37      0.14     0.12     0.66 1.00     7834     5933
```

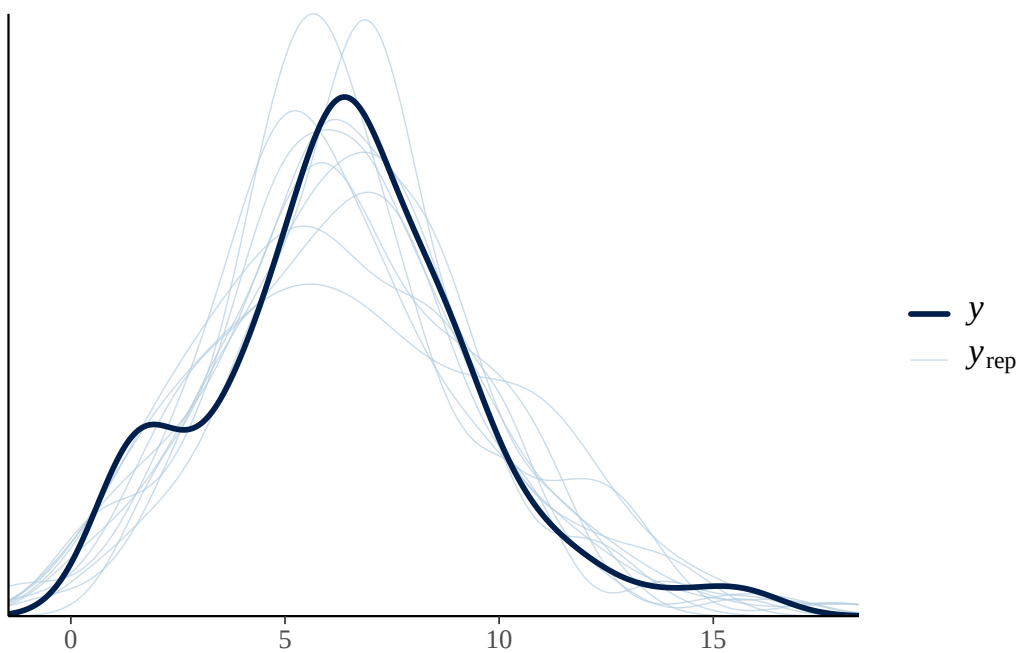
```
Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     2.27      0.18     1.94     2.66 1.00     4932     5066
alpha     5.78      1.86     2.90    10.14 1.00     5487     3668

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
bayes_R2(model_pca_skew) #check R^2
```

```
   Estimate  Est.Error      Q2.5     Q97.5
R2 0.409568 0.04208504 0.3203448 0.4865482
```

```r
#posterior predictive check
pp_check(model_pca_skew)
```

```
Using 10 posterior draws for ppc type 'dens_overlay' by default.
```



```r
#compare gaussian and skew_normal models --> model_pca_skew seems to be the better model!
loo_gaussian <- loo(model_pca_gaus, moment_match = TRUE)
loo_skew <- loo(model_pca_skew, reloo = TRUE)
```

```
No problematic observations found. Returning the original 'loo' object.
```

```r
loo_compare(loo_gaussian, loo_skew)
```

```
               elpd_diff se_diff
model_pca_skew   0.0       0.0
model_pca_gaus -11.9       2.7
```

### Step 4: Interpreting and Visualizing Model

Now that we have our model, we can start to interpret and visualize it. Starting with the PCs, we created a for loop to print the variables associated with each significant PC.

#### PC Interpretations

PC1, PC2, and PC3 were all significant predictors of purple urchin gonad index in the Bayesian skew-normal model (all 95% CIs excluded zero, Rhats = 1.00).

PC1 was the strongest predictor of purple urchin gonad index ( = 0.80, 95% CI: 0.62–0.96) and represented a gradient from kelp forest to barren-like habitat. Zones scoring high on PC1 were characterized by greater *Macrocystis* holdfast cover, stipe density, macro-algal abundance, and concealed purple urchins (meaning urchins were eating drift kelp rather than live kelp, therefore not contributing to kelp loss). In contrast, zones scoring low on PC1 were characterized by higher crustose coralline algae cover, marine snail density, exposed red and purple urchin densities, and lower substrate relief, which can be indicators of degraded, barren-like habitat. The positive model coefficient indicates that urchins at kelp forest sites had significantly higher gonad index than those at barren-like sites, consistent with drift kelp being the primary food source supporting urchin gonad development.

PC2 had a negative effect ( = -0.58, 95% CI: -0.82– -0.35), suggesting that sites with higher red urchin density, *Corynactis*, and barnacle cover were associated with lower gonad index compared to sites with articulated coralline algae and *Desmarestia*. PC3 had a moderate positive effect ( = 0.37, 95% CI: 0.13–0.67), where sites with greater algal diversity were associated with higher gonad index relative to sites dominated by *Laminaria* stumps, purple urchin densities, and dead kelp holdfasts.

Together, these three habitat gradients explained 41% of variation in purple urchin gonad index (Bayesian $R^2 = 0.41$, 95% CI: 0.32–0.49).

```r
#what variables are included in PC1, PC2, and PC3?
for(pc in c("PC1","PC2","PC3")) { #starts a for loop
  cat("\n---", pc, "---\n") #syntax for output: headers for each PC
  cat("TOP POSITIVE:\n") #syntax for output: label for the variables with  largest positive l
  print(head(sort(pca$rotation[,pc], decreasing = TRUE), 5)) #sorts loadings from largest to
  cat("TOP NEGATIVE:\n") #syntax for output: label for the variables with  largest negative l
  print(head(sort(pca$rotation[,pc], decreasing = FALSE), 5))} #sorts loadings from largest t
```

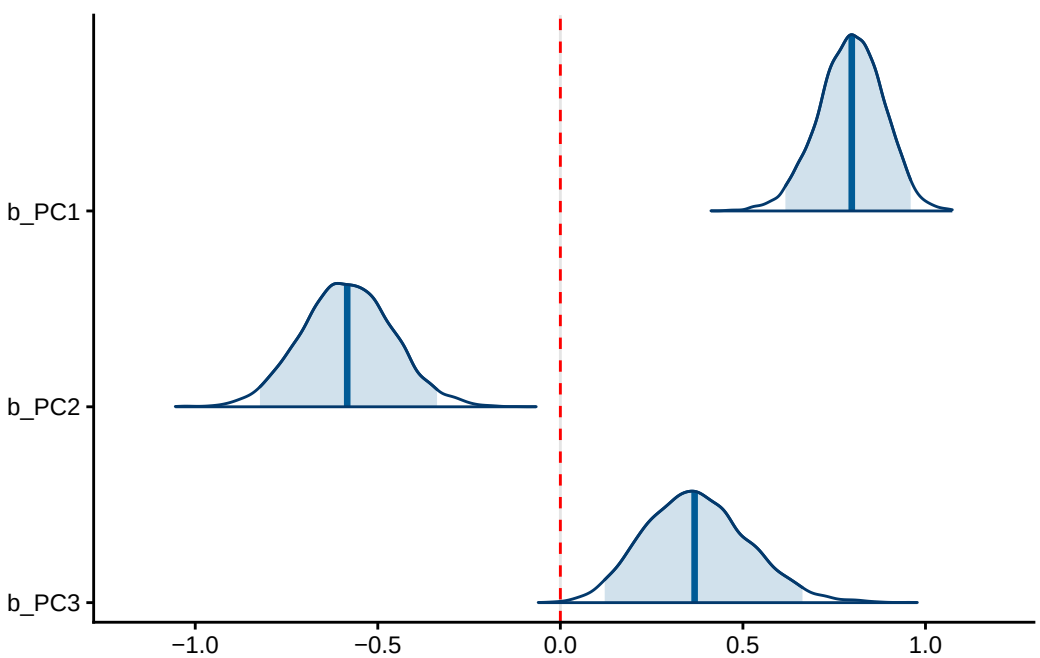
```
--- PC1 ---
TOP POSITIVE:
     cov_mac_holdfast_live     macro_stipe_density_m2
                 0.3680964                  0.3574166
         n_macro_plants_m2  purple_urchin_conceiledm2
                 0.3373214                  0.2761511
cov_dead_kelp_holdfast_any
                 0.2658364
TOP NEGATIVE:
 cov_crustose_coralline           marine_snails    red_urchin_densitym2
             -0.3082244              -0.2965373              -0.2233733
              relief_cm purple_urchin_densitym2
             -0.1767785              -0.1667453

--- PC2 ---
TOP POSITIVE:
cov_corynactis_californica       red_urchin_densitym2
                 0.3731614                  0.3651667
              cov_barnacle              cov_bare_sand
                 0.2918554                  0.2742160
         n_macro_plants_m2
                 0.2156306
TOP NEGATIVE:
cov_articulated_coralline       cov_desmarestia_spp purple_urchin_conceiledm2
               -0.3697764                -0.2912846                -0.2783927
            marine_snails                 relief_cm
               -0.2404735                -0.2300027

--- PC3 ---
TOP POSITIVE:
cov_articulated_coralline             cov_bare_sand         n_macro_plants_m2
                0.2013814                 0.1504267                 0.1436898
      cov_desmarestia_spp      cov_dictyoneurum_spp
```

```
                0.1365287                  0.1265713
TOP NEGATIVE:
          density_lamstump    purple_urchin_densitym2
                -0.4460719                 -0.4396316
cov_dead_kelp_holdfast_any  purple_urchin_conceiledm2
                -0.3414915                 -0.3259312
      red_urchin_densitym2
                -0.2715089
```

### Visualizations

The

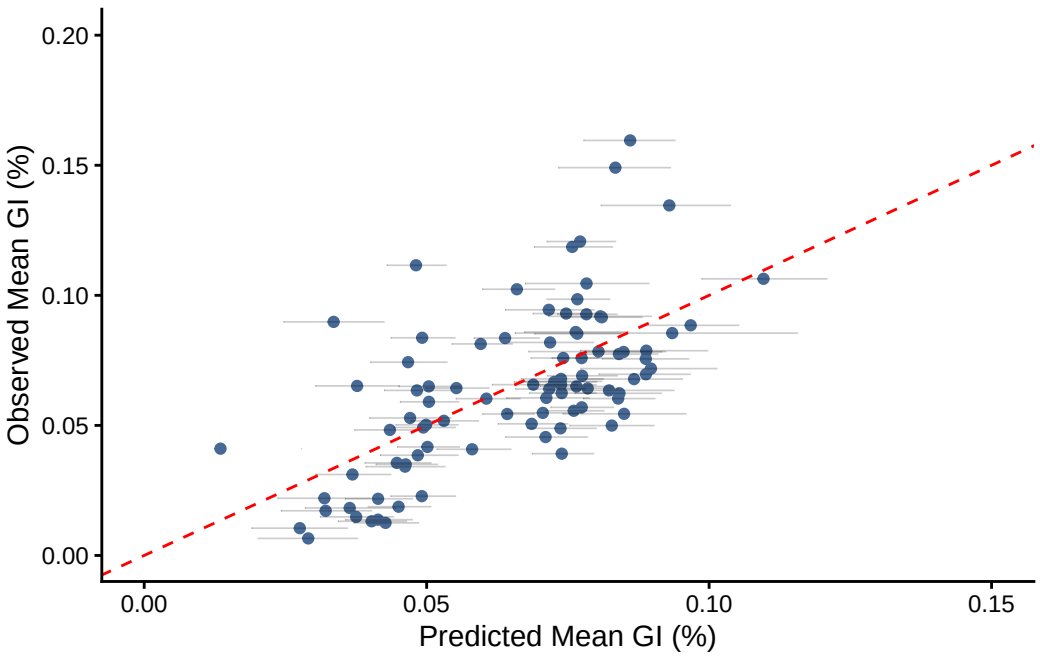
```
#density curves of each PC showing posterior distributions
mcmc_areas(model_pca_skew,
           pars = vars(starts_with("b_"), -b_Intercept), #plot regression coefficients and ex
           prob = 0.95, #shade the 95% credible interval
           prob_outer = 1.0) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "red") + #dotted line at x=0 for vi
  theme_classic()
```




explain predicted vs observation plot

```r
pc_data$fitted <- fitted(model_pca_skew)[, "Estimate"] #posterior mean prediction for each ob
pc_data$lower  <- fitted(model_pca_skew)[, "Q2.5"] #lower 95% credible interval
pc_data$upper  <- fitted(model_pca_skew)[, "Q97.5"] #upper 95% credible interval

#plot observed vs predicted to see how well the three ecologically meaningful PCs predict gor
ggplot(pc_data, aes(x = fitted/100, y = mean_gi/100)) + #turn units to %
  geom_point(color = "#466995") + #each dot is one observation
  geom_errorbarh(aes(xmin = lower/100, xmax = upper/100),
                 alpha = 0.2,
                 linewidth = 0.3) + #horizontal error bars showing prediction uncertainty
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red") + #prediction lin
  labs(x = "Predicted Mean GI (%)", y = "Observed Mean GI (%)") +
  scale_x_continuous(limits = c(0,0.15)) +
  scale_y_continuous(limits = c(0,0.2)) +
  theme_classic()
```

```
Warning: `geom_errorbarh()` was deprecated in ggplot2 4.0.0.
i Please use the `orientation` argument of `geom_errorbar()` instead.
```

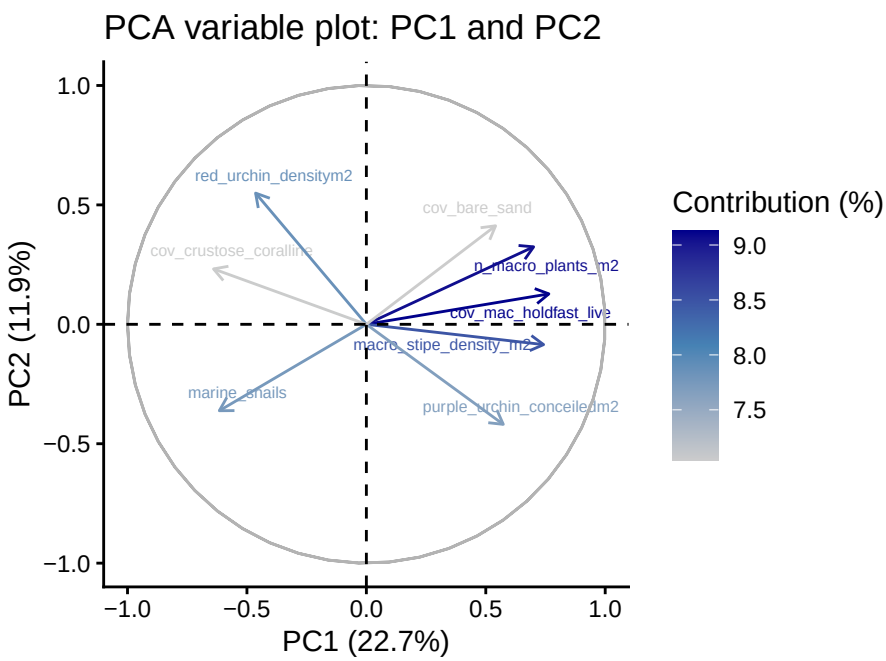



```r
summary(pca)$importance[2, 1:3] * 100 # % variance for each PC
```

```
   PC1    PC2    PC3
22.686 11.911 10.193
```

```r
#plotting PC1 and PC2
fviz_pca_var(pca, axes = c(1, 2),#specify which PCs to plot
             select.var = list(contrib = 8),  #only top # contributors
             col.var = "contrib",
             repel = TRUE,
               labelsize = 2,
             gradient.cols = c("grey80", "steelblue", "darkblue")) +
  labs(title = "PCA variable plot: PC1 and PC2",
       x = "PC1 (22.7%)",
       y = "PC2 (11.9%)",
       color = "Contribution (%)")+
  theme_classic()
```

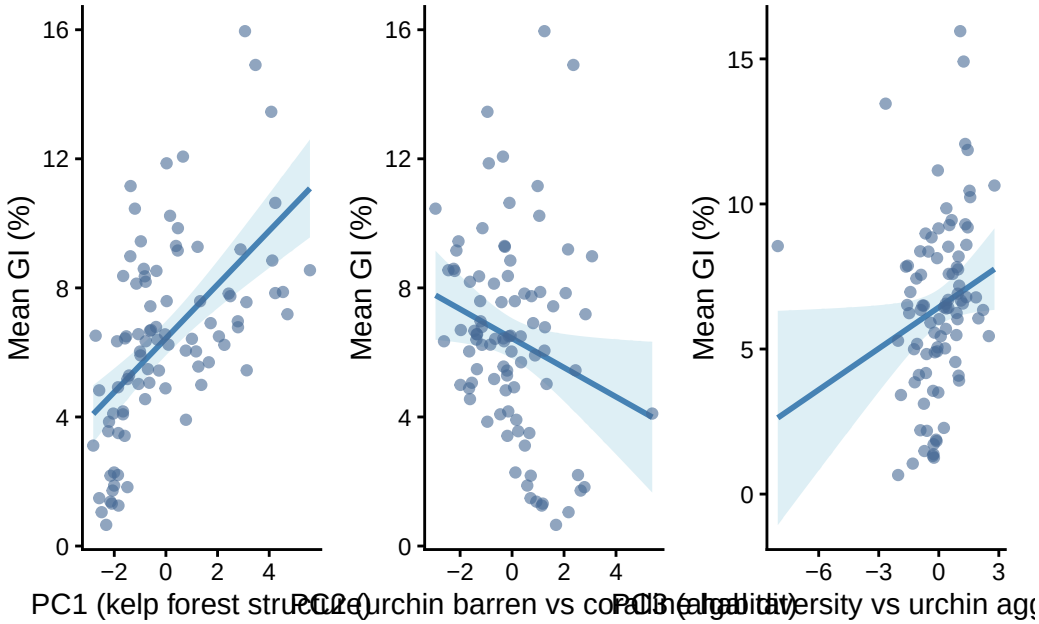
```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
i The deprecated feature was likely used in the ggpubr package.
  Please report the issue at <https://github.com/kassambara/ggpubr/issues>.

Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
i Please use tidy evaluation idioms with `aes()`.
i See also `vignette("ggplot2-in-packages")` for more information.
i The deprecated feature was likely used in the factoextra package.
  Please report the issue at <https://github.com/kassambara/factoextra/issues>.
```




```
#add individual plots for each PC
# Plot for PC1
p1 <- pc_data %>%
  ggplot(aes(x = PC1, y = mean_gi)) +
  stat_smooth(method = "lm", color = "steelblue", fill = "lightblue") +
  geom_point(color = "#466995", alpha = 0.6) +
  labs(x = "PC1 (kelp forest structure)",
       y = "Mean GI (%)") +
  theme_classic()

# Plot for PC2
p2 <- pc_data %>%
  ggplot(aes(x = PC2, y = mean_gi)) +
  stat_smooth(method = "lm", color = "steelblue", fill = "lightblue") +
  geom_point(color = "#466995", alpha = 0.6) +
  labs(x = "PC2 (urchin barren vs coralline habitat)",
       y = "Mean GI (%)") +
  theme_classic()

# Plot for PC3
p3 <- pc_data %>%
  ggplot(aes(x = PC3, y = mean_gi)) +
  stat_smooth(method = "lm", color = "steelblue", fill = "lightblue") +
```

```
  geom_point(color = "#466995", alpha = 0.6) +
  labs(x = "PC3 (algal diversity vs urchin aggregation)",
       y = "Mean GI (%)") +
  theme_classic()

# Combine all three
p1 + p2 + p3
```

```
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
```




## 4. uhhhh - horseshoe?

```
#explain why hand-picking is not as honest as PCA?
glm <- glm(mean_gi ~ PC1 + PC3 + PC6, data = pc_data)


#HORSESHOE
```

```r
#scale predictors
predictors_scaled <- predictors %>%
  mutate(across(-mean_gi, scale))

#model with horseshoe instead of pca
brm_hs <- brm(
  mean_gi ~ .,
  data = predictors_scaled,
  prior = prior(horseshoe(1), class = "b"),
  control = list(adapt_delta = 0.99))
```

```
Compiling Stan program...

Trying to compile a simple C file

Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
using C compiler: ‘Apple clang version 17.0.0 (clang-1700.6.3.2)’
using SDK: ‘MacOSX26.2.sdk’
clang -arch arm64 -std=gnu2x -I"/Library/Frameworks/R.framework/Resources/include" -DNDEBUG
In file included from <built-in>:1:
In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/St
In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/Rc
In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/Rc
/Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/RcppEigen/include/Eigen/
  679 | #include <cmath>
      |          ^~~~~~~
1 error generated.
make: *** [foo.o] Error 1

Start sampling


SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 5.1e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.51 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
Chain 1: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 1: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 1: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 1: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 1: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 1: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 1: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 1: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 1: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 1: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.885 seconds (Warm-up)
Chain 1:                0.74 seconds (Sampling)
Chain 1:                1.625 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 7e-06 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.07 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 2: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 2: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 2: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 2: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 2: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 2: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 2: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 2: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 2: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 2: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.93 seconds (Warm-up)
Chain 2:                0.727 seconds (Sampling)
Chain 2:                1.657 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
```

```
Chain 3:
Chain 3: Gradient evaluation took 7e-06 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.07 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 3: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 3: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 3: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 3: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 3: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 3: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 3: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 3: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 3: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 3: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 3: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.854 seconds (Warm-up)
Chain 3:                1.242 seconds (Sampling)
Chain 3:                2.096 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 7e-06 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.07 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 4: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 4: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 4: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 4: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 4: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 4: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 4: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 4: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 4: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 4: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 4: Iteration: 2000 / 2000 [100%]  (Sampling)
```

```
Chain 4:
Chain 4:  Elapsed Time: 0.92 seconds (Warm-up)
Chain 4:                0.74 seconds (Sampling)
Chain 4:                1.66 seconds (Total)
Chain 4:
```

```r
summary(brm_hs)
```

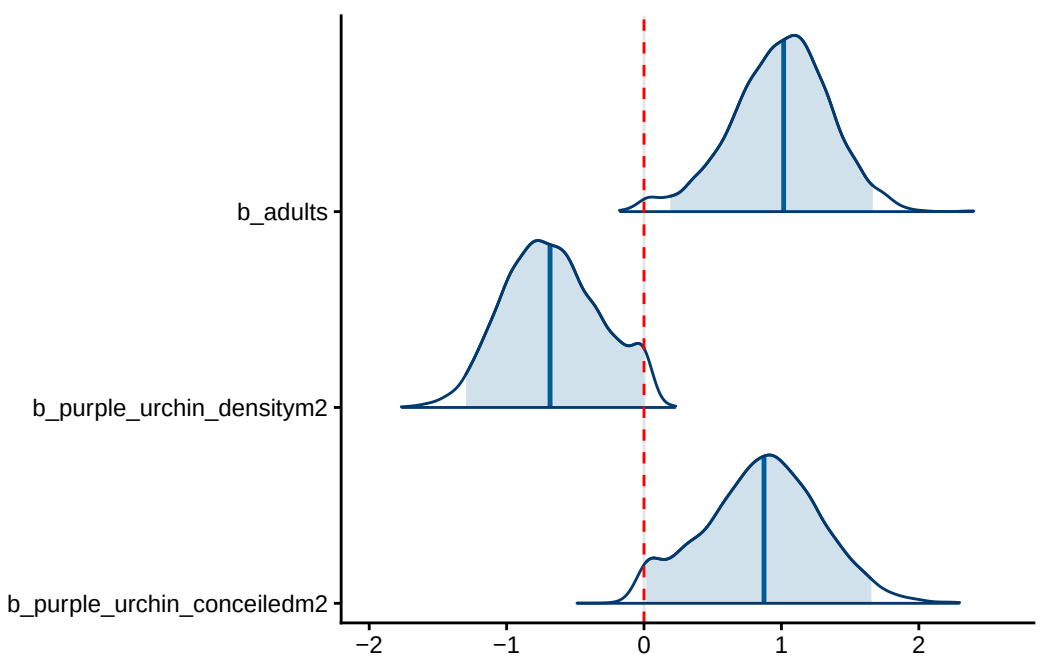
```
 Family: gaussian
  Links: mu = identity
Formula: mean_gi ~ n_macro_plants_m2 + macro_stipe_density_m2 + adults + juveniles + density_
   Data: predictors_scaled (Number of observations: 90)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
                           Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
Intercept                      6.43      0.21     6.03     6.84 1.00     6723
n_macro_plants_m2              0.12      0.21    -0.20     0.65 1.00     4059
macro_stipe_density_m2        -0.07      0.20    -0.59     0.28 1.00     3788
adults                         0.99      0.36     0.19     1.66 1.00     3200
juveniles                     -0.05      0.25    -0.67     0.46 1.00     3691
density_cancer                 0.04      0.15    -0.24     0.39 1.00     5225
density_lamstump              -0.18      0.26    -0.82     0.18 1.00     3065
density_macstump               0.00      0.15    -0.31     0.32 1.00     5842
relief_cm                     -0.06      0.17    -0.47     0.23 1.00     4747
risk_index                    -0.26      0.26    -0.80     0.09 1.00     2310
purple_urchin_densitym2       -0.67      0.35    -1.30     0.00 1.00     1876
purple_urchin_conceiledm2      0.86      0.42     0.02     1.65 1.00     2075
red_urchin_densitym2          -0.11      0.23    -0.69     0.28 1.00     3515
red_urchin_conceiledm2        -0.06      0.15    -0.44     0.22 1.00     4842
marine_snails                  0.02      0.16    -0.30     0.38 1.00     5179
lamr                          -0.44      0.35    -1.17     0.06 1.00     2161
macr                           0.26      0.25    -0.09     0.82 1.00     3056
cov_crustose_coralline        -0.29      0.34    -1.11     0.14 1.00     2617
cov_desmarestia_spp            0.06      0.16    -0.21     0.47 1.00     4414
cov_lam_holdfast_live          0.03      0.22    -0.41     0.57 1.00     4180
cov_articulated_coralline      0.10      0.18    -0.17     0.56 1.00     3974
cov_fleshy_red                 0.15      0.22    -0.18     0.67 1.00     3501
cov_bare_rock                  0.00      0.14    -0.31     0.32 1.00     5440
cov_green_algae                0.04      0.14    -0.23     0.38 1.00     5512
cov_barnacle                   0.52      0.29    -0.01     1.08 1.00     2361
```

| | | | | | | |
|---|---|---|---|---|---|---|
| cov_bare_sand | -0.03 | 0.17 | -0.44 | 0.31 | 1.00 | 5278 |
| cov_colonial_tunicate | 0.05 | 0.17 | -0.26 | 0.46 | 1.00 | 4768 |
| cov_dictyota_dictyopteris | 0.05 | 0.15 | -0.21 | 0.40 | 1.00 | 5154 |
| cov_sponge | -0.01 | 0.15 | -0.34 | 0.32 | 1.00 | 4799 |
| cov_dead_kelp_holdfast_any | 0.01 | 0.17 | -0.34 | 0.38 | 1.00 | 5351 |
| cov_mac_holdfast_live | 0.03 | 0.18 | -0.32 | 0.46 | 1.00 | 4628 |
| cov_red_turf_2_cm | -0.14 | 0.20 | -0.62 | 0.13 | 1.00 | 2583 |
| cov_corynactis_californica | -0.29 | 0.26 | -0.84 | 0.07 | 1.00 | 2697 |
| cov_dictyoneurum_spp | 0.03 | 0.14 | -0.25 | 0.34 | 1.00 | 5207 |

| | Tail_ESS |
|---|---|
| Intercept | 3124 |
| n_macro_plants_m2 | 3600 |
| macro_stipe_density_m2 | 3485 |
| adults | 2650 |
| juveniles | 3610 |
| density_cancer | 3641 |
| density_lamstump | 3783 |
| density_macstump | 4103 |
| relief_cm | 3745 |
| risk_index | 3211 |
| purple_urchin_densitym2 | 1198 |
| purple_urchin_conceiledm2 | 1393 |
| red_urchin_densitym2 | 3723 |
| red_urchin_conceiledm2 | 3621 |
| marine_snails | 3757 |
| lamr | 3287 |
| macr | 3171 |
| cov_crustose_coralline | 3268 |
| cov_desmarestia_spp | 3445 |
| cov_lam_holdfast_live | 3250 |
| cov_articulated_coralline | 3944 |
| cov_fleshy_red | 3361 |
| cov_bare_rock | 4028 |
| cov_green_algae | 3835 |
| cov_barnacle | 2074 |
| cov_bare_sand | 3868 |
| cov_colonial_tunicate | 3780 |
| cov_dictyota_dictyopteris | 3815 |
| cov_sponge | 3842 |
| cov_dead_kelp_holdfast_any | 3763 |
| cov_mac_holdfast_live | 3483 |
| cov_red_turf_2_cm | 3291 |
| cov_corynactis_californica | 3438 |

```
cov_dictyoneurum_spp           3405

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     2.03      0.18     1.70     2.42 1.00     2341     2712

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
#select only significant variables
significant_vars <- c("b_adults",
                      "b_purple_urchin_densitym2",
                      "b_purple_urchin_conceiledm2")
mcmc_areas(brm_hs,
           pars = significant_vars,
           prob = 0.95) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "red") +
  theme_classic()
```



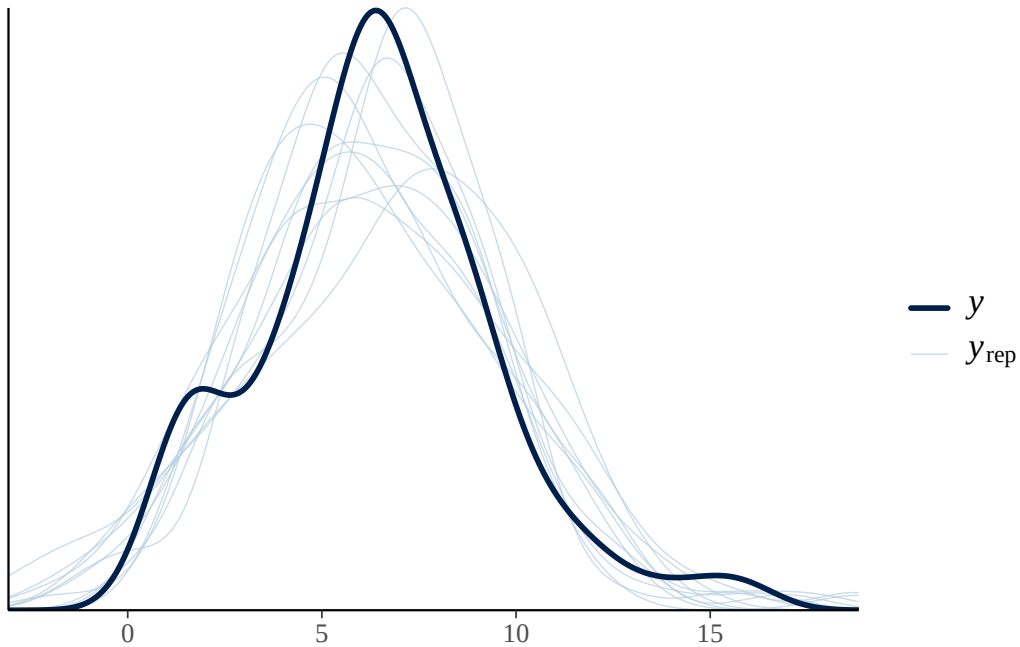

```r
fixef(brm_hs)
```

```
                                Estimate Est.Error          Q2.5       Q97.5
```

```
Intercept                   6.4317350425 0.2093999  6.027109862 6.839778625
n_macro_plants_m2           0.1161986349 0.2112975 -0.198077351 0.647247275
macro_stipe_density_m2     -0.0697632225 0.2046684 -0.594131331 0.280909956
adults                      0.9938541603 0.3577363  0.190313736 1.664929942
juveniles                  -0.0548821254 0.2533315 -0.670105476 0.458667726
density_cancer              0.0412395144 0.1456109 -0.236132392 0.390483600
density_lamstump           -0.1777798499 0.2567263 -0.815105775 0.178838124
density_macstump            0.0003933328 0.1452035 -0.312335445 0.323334030
relief_cm                  -0.0594186760 0.1663999 -0.471086282 0.230031433
risk_index                 -0.2588951591 0.2557109 -0.804883334 0.092449095
purple_urchin_densitym2    -0.6670632315 0.3477147 -1.295171590 0.004788707
purple_urchin_conceiledm2   0.8561002438 0.4227458  0.016468360 1.654699643
red_urchin_densitym2       -0.1074582776 0.2322189 -0.693251646 0.281032016
red_urchin_conceiledm2     -0.0569208339 0.1540814 -0.439825665 0.223885731
marine_snails               0.0208745280 0.1559715 -0.299657138 0.384866497
lamr                       -0.4430767997 0.3526604 -1.173583374 0.064535243
macr                        0.2555421008 0.2511500 -0.088332190 0.817540916
cov_crustose_coralline     -0.2896562942 0.3430724 -1.113117227 0.143713504
cov_desmarestia_spp         0.0586859121 0.1592411 -0.214636048 0.465808747
cov_lam_holdfast_live       0.0286554448 0.2218053 -0.405271438 0.574066302
cov_articulated_coralline   0.0997171015 0.1817770 -0.174500316 0.558596678
cov_fleshy_red              0.1460848035 0.2179208 -0.177337495 0.672134458
cov_bare_rock               0.0023853382 0.1391308 -0.306163027 0.315993437
cov_green_algae             0.0389983298 0.1446251 -0.233805066 0.380954595
cov_barnacle                0.5238354451 0.2924490 -0.007247128 1.081334641
cov_bare_sand              -0.0307945418 0.1679880 -0.438480887 0.313030083
cov_colonial_tunicate       0.0457478116 0.1652776 -0.262686273 0.460416723
cov_dictyota_dictyopteris   0.0520195388 0.1489592 -0.213275671 0.400656309
cov_sponge                 -0.0055923722 0.1468194 -0.337904568 0.319260110
cov_dead_kelp_holdfast_any  0.0097176888 0.1664286 -0.344754694 0.377087704
cov_mac_holdfast_live       0.0329291470 0.1810560 -0.323545114 0.463726048
cov_red_turf_2_cm          -0.1429868147 0.1972870 -0.621566948 0.128187294
cov_corynactis_californica -0.2938088519 0.2583431 -0.844166038 0.067097098
cov_dictyoneurum_spp        0.0272841288 0.1357943 -0.246175891 0.340764839
```

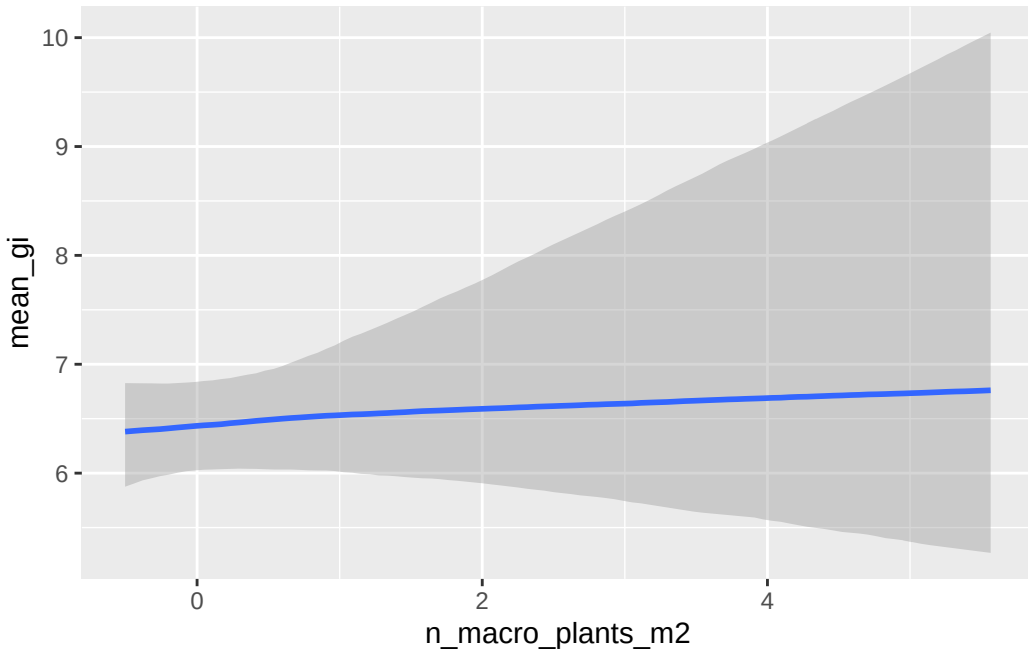
```r
pp_check(brm_hs, pars = significant_vars)
```

```
Using 10 posterior draws for ppc type 'dens_overlay' by default.

Warning: The following arguments were unrecognized and ignored: pars
```

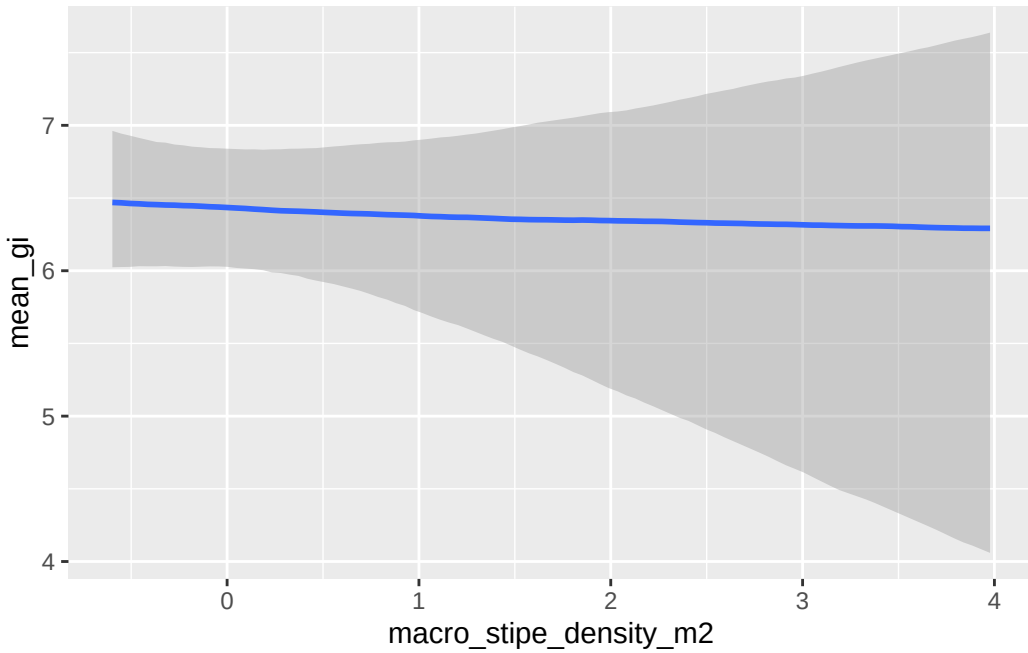



```r
conditional_effects(brm_hs)
```

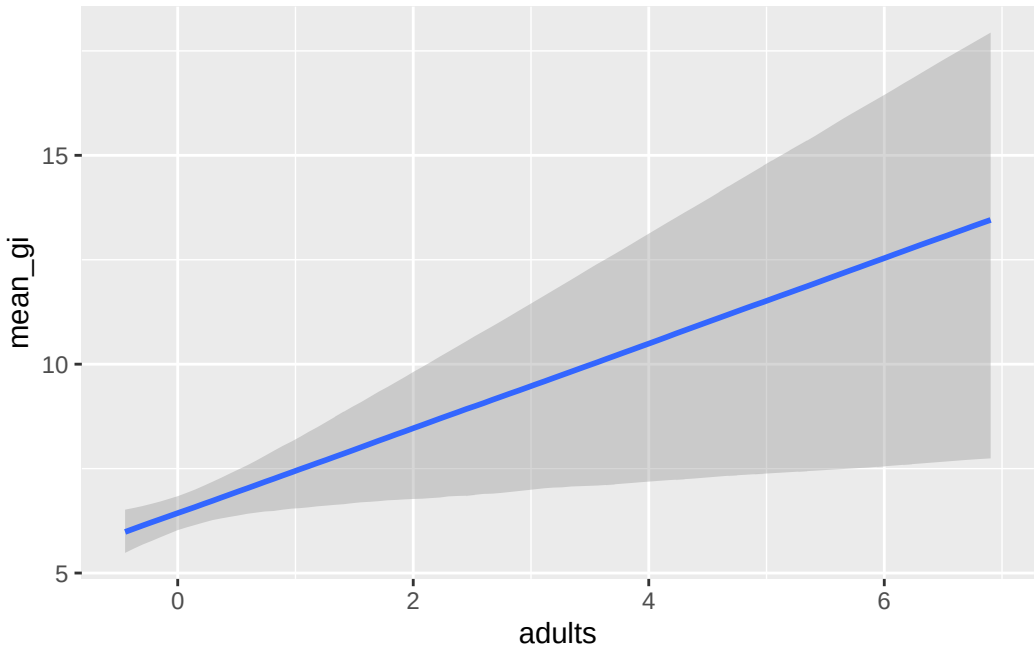
```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"

Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



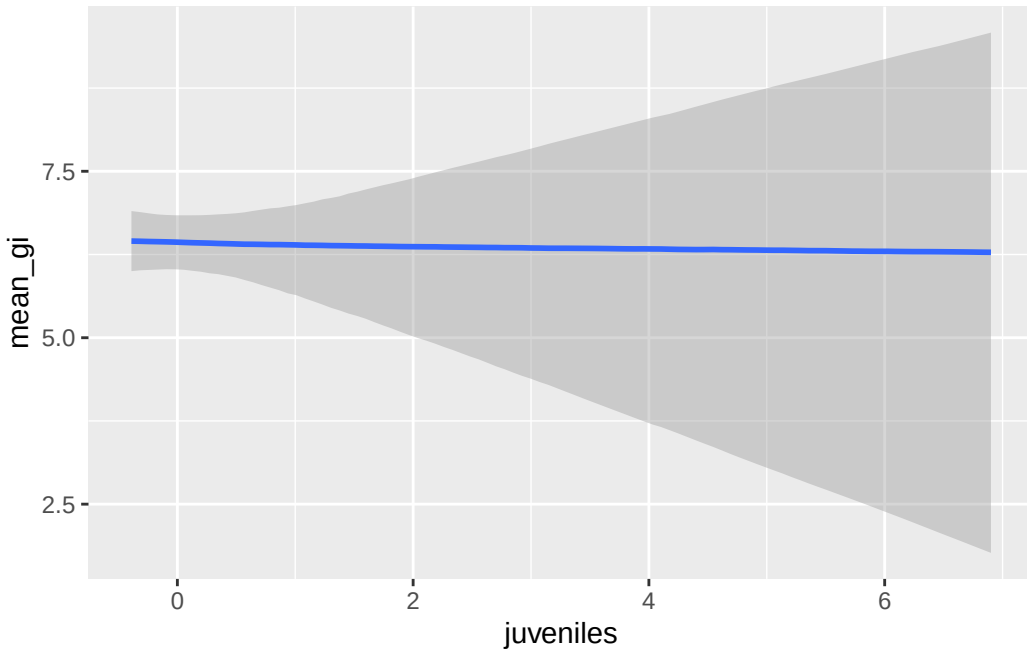

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



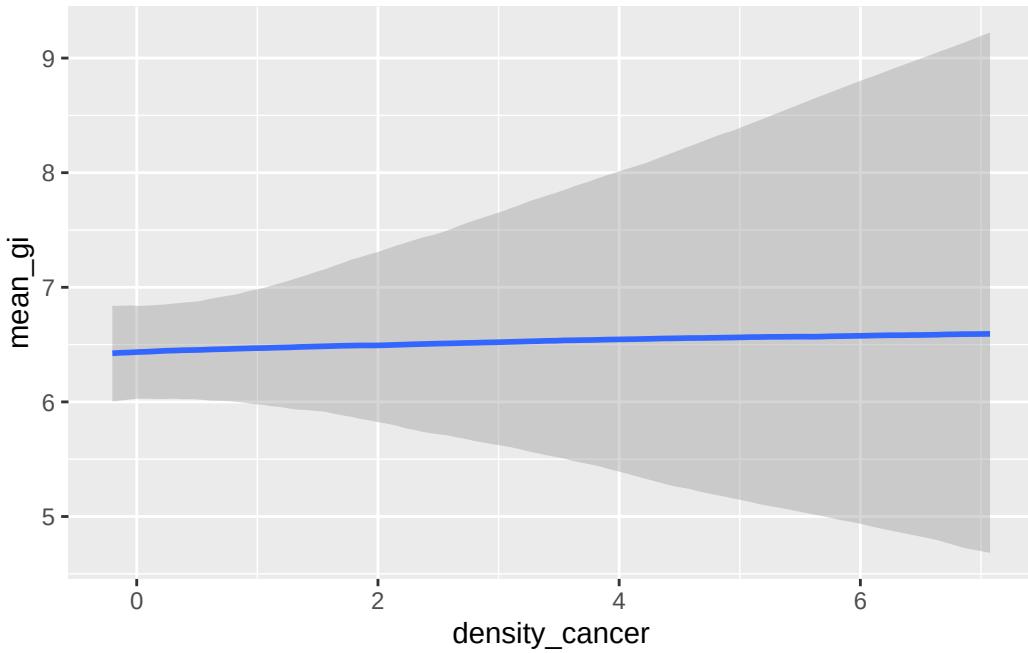

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



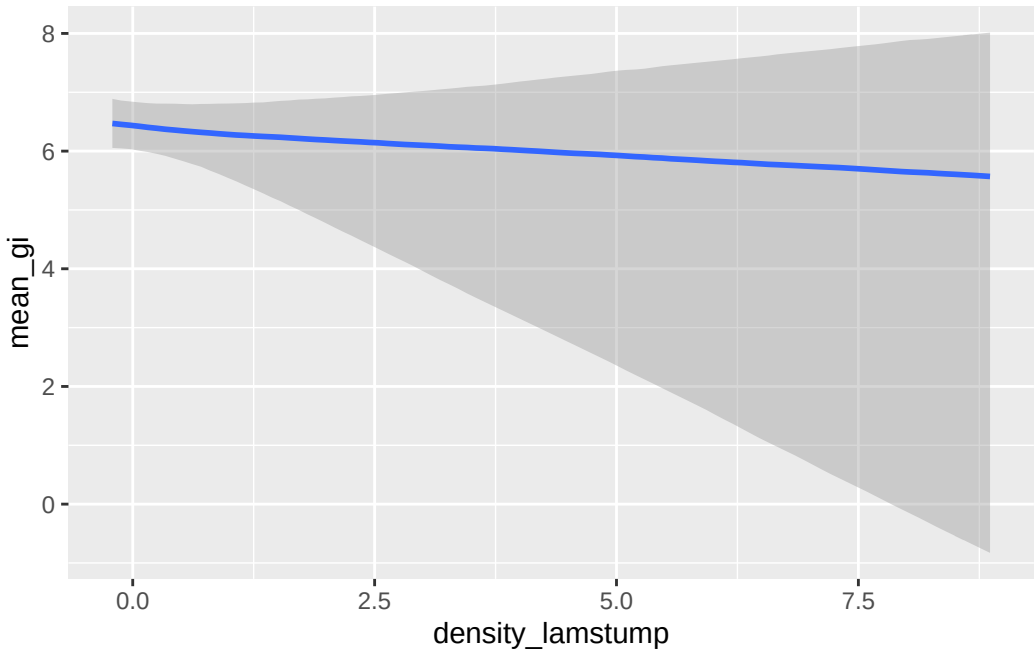

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



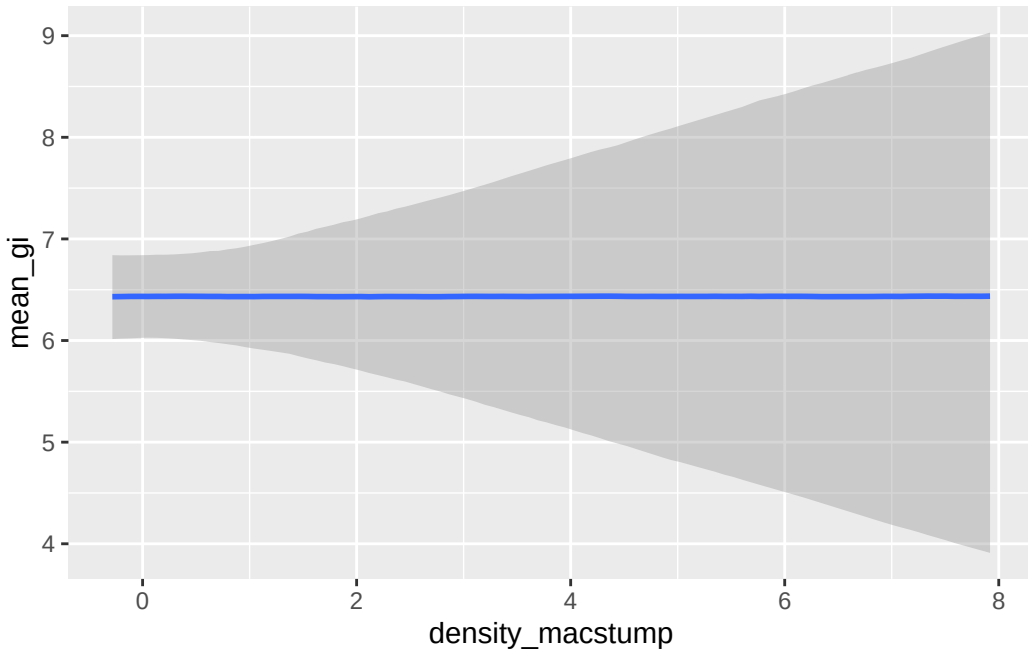

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



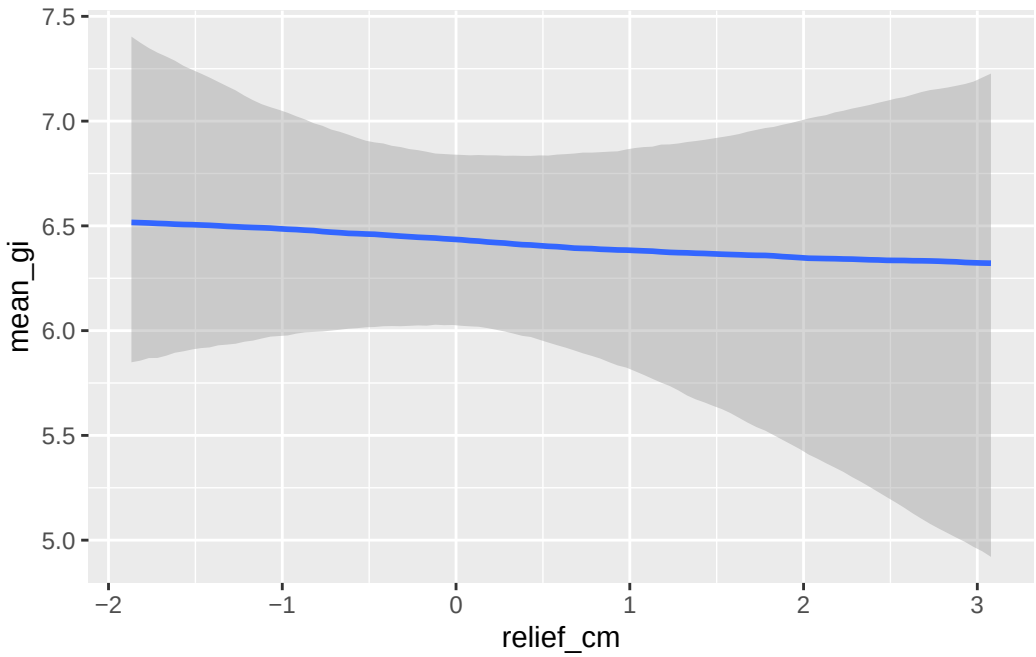

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



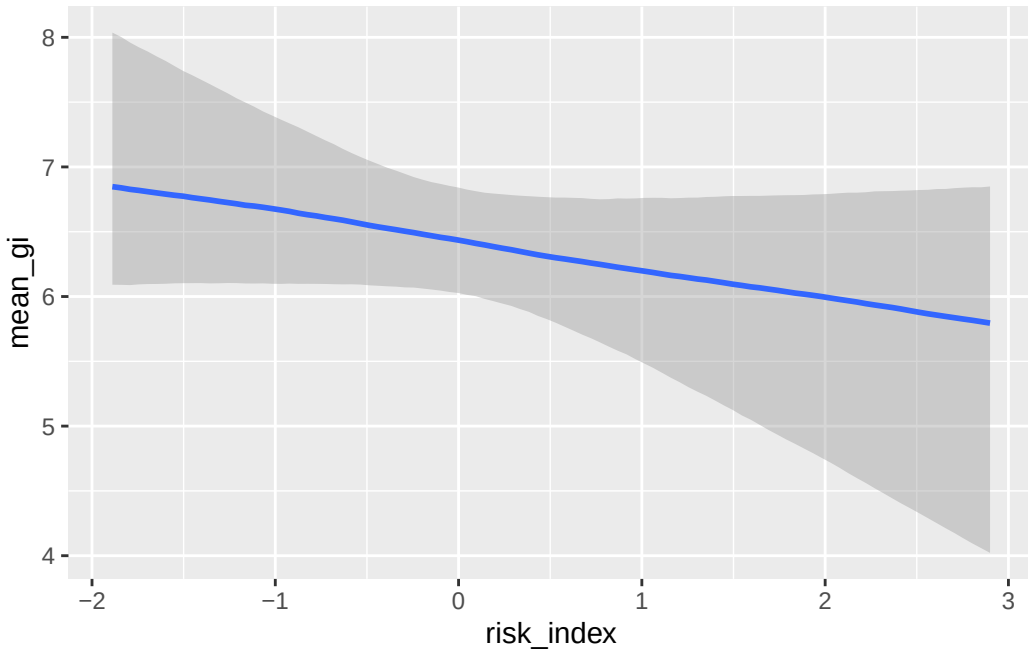

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



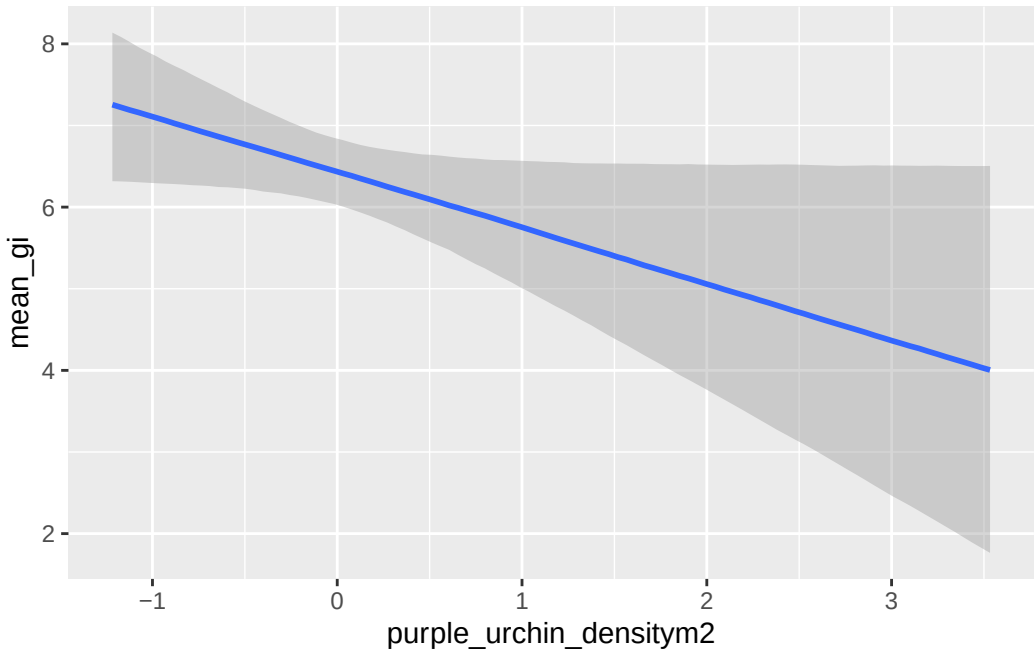

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



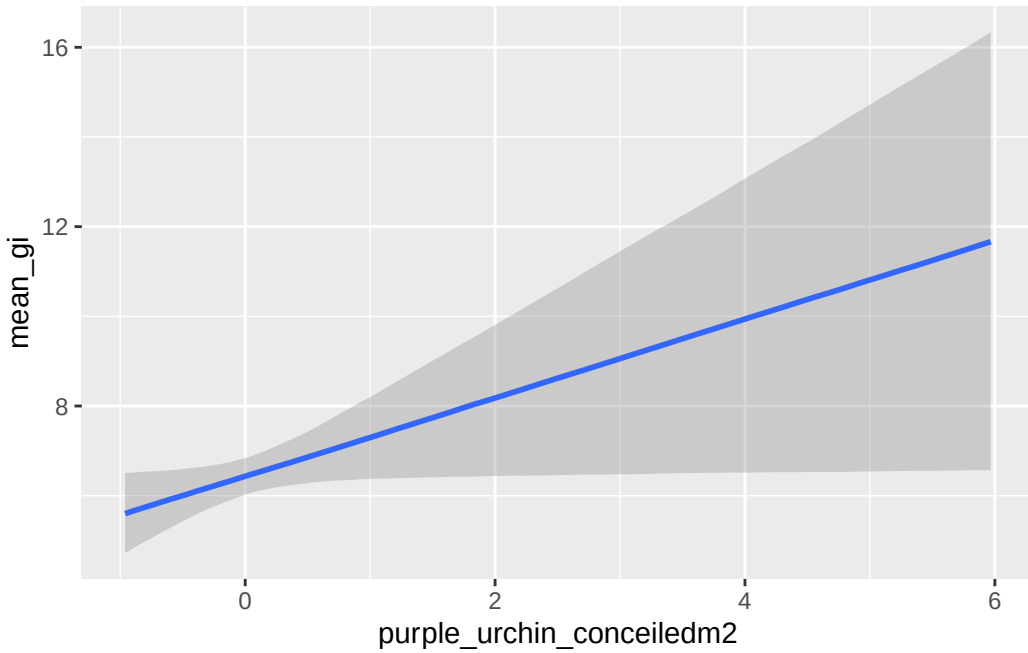

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



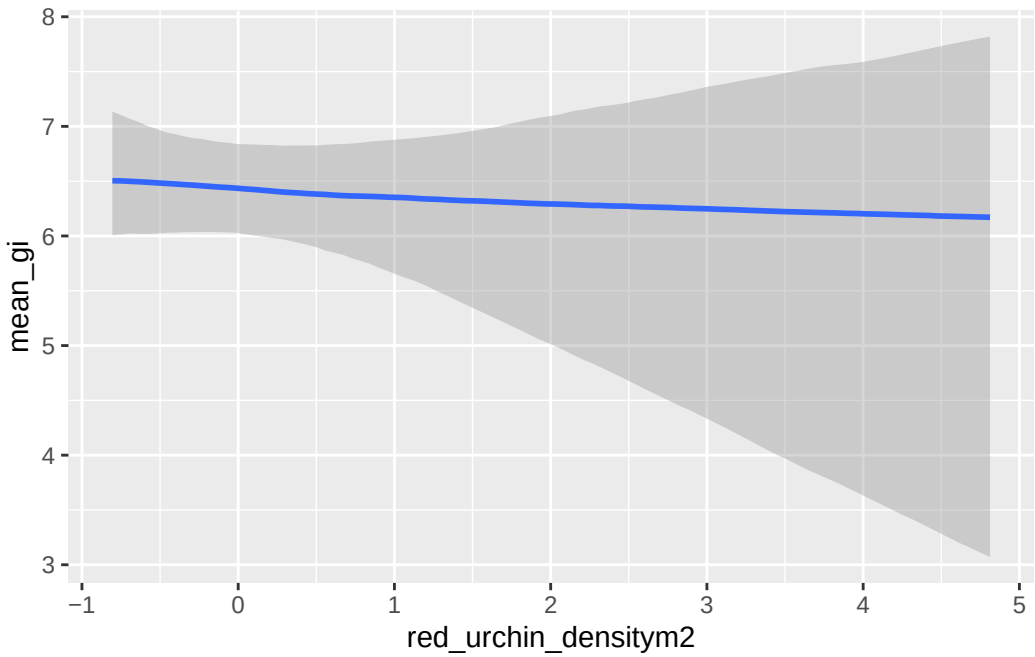

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



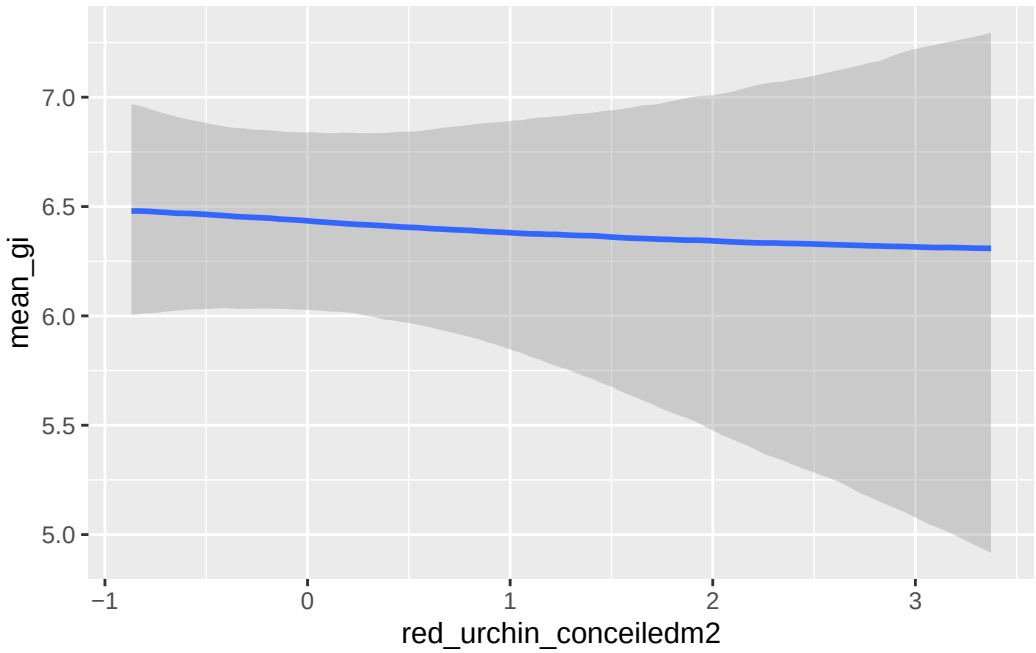

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



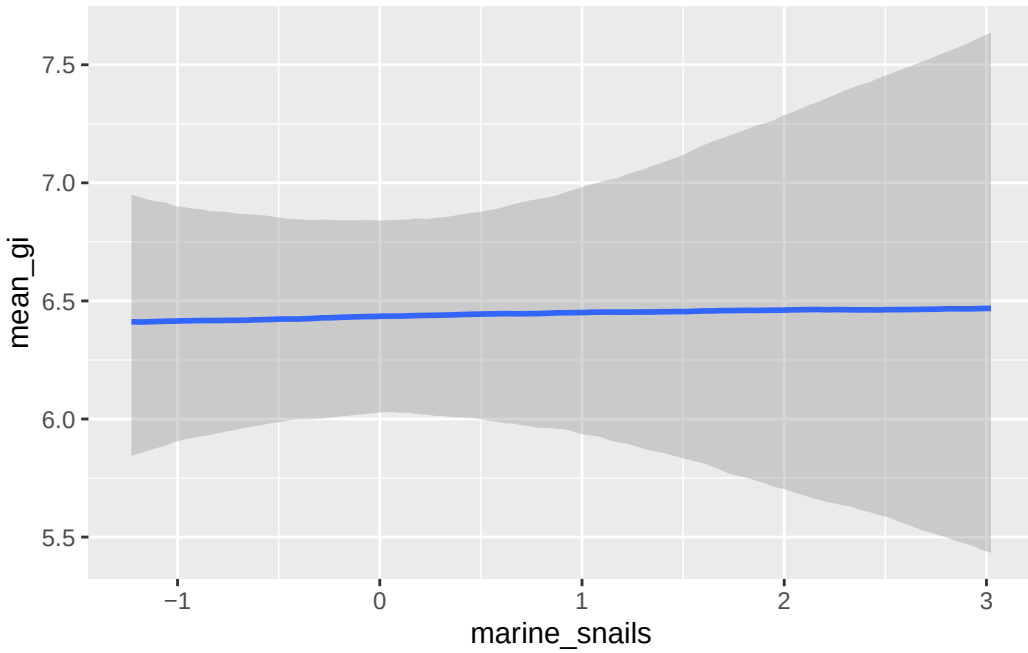

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



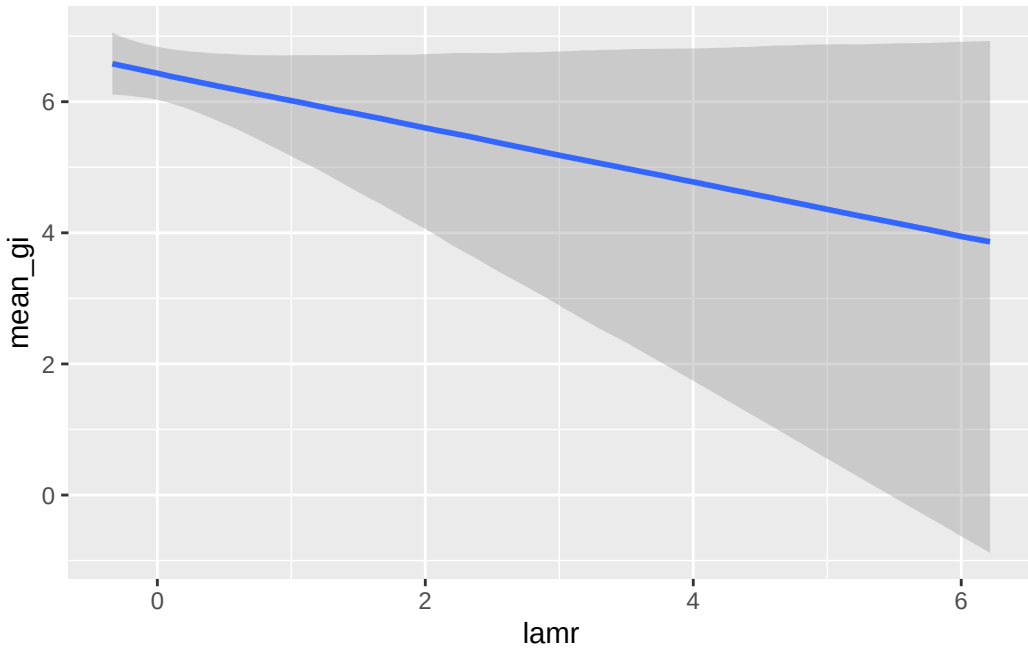

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



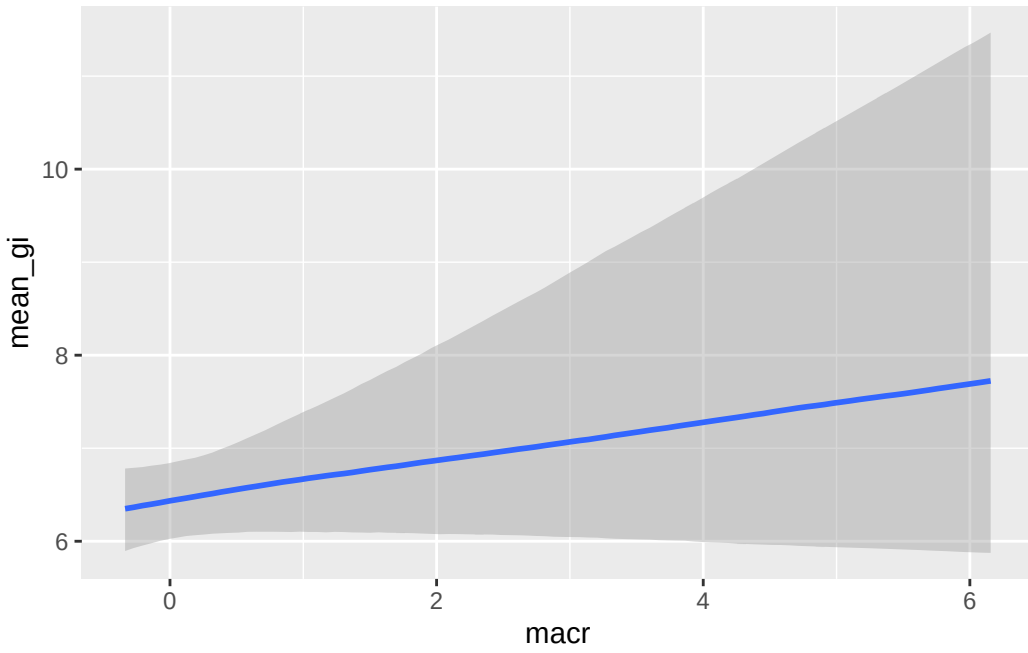

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



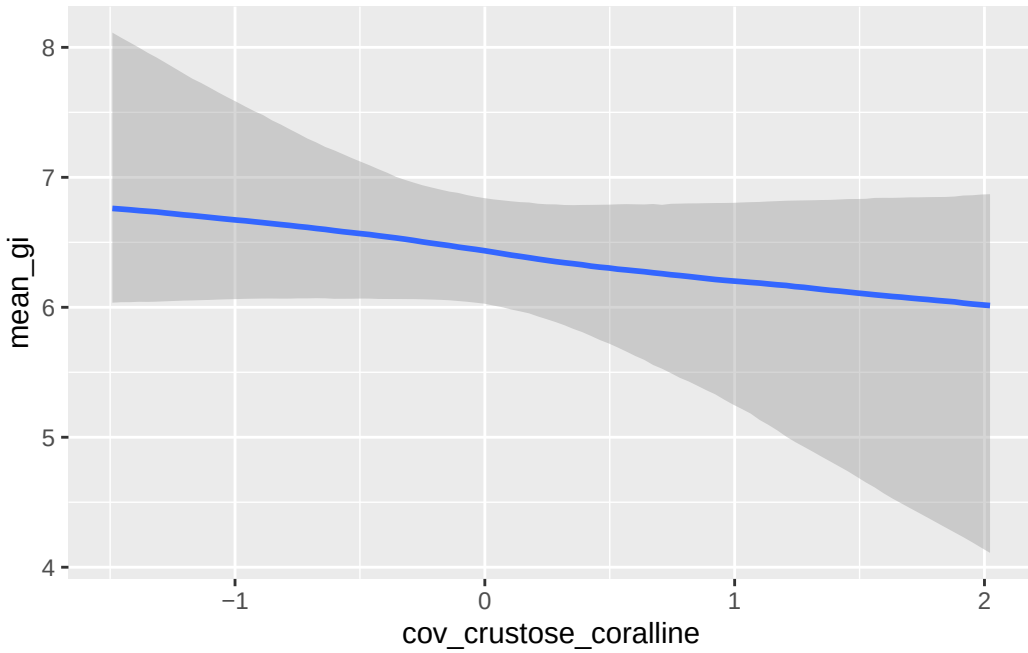

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



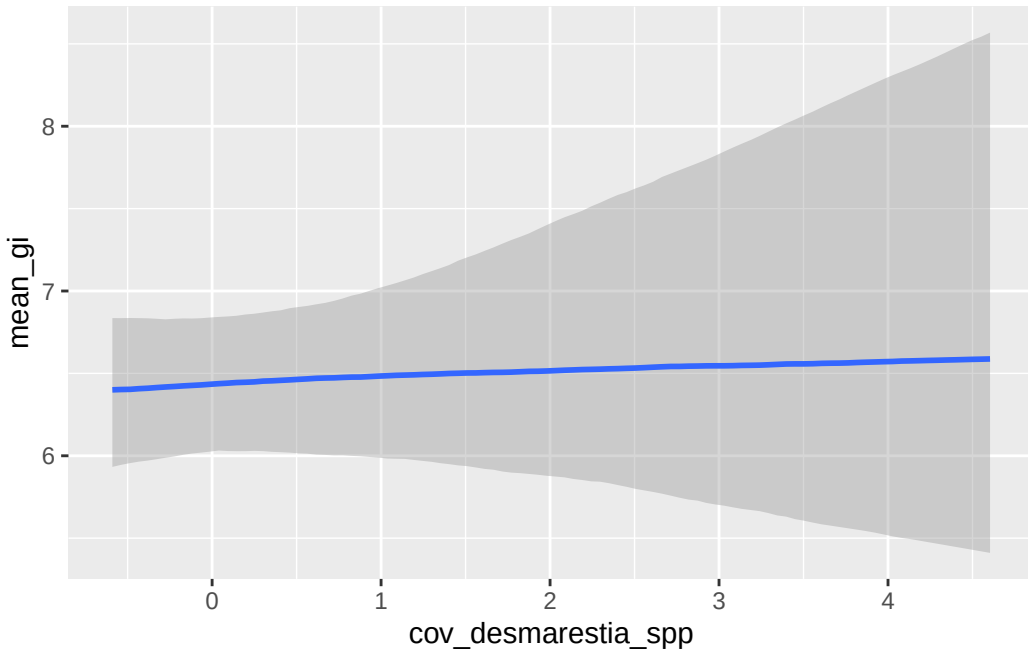

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



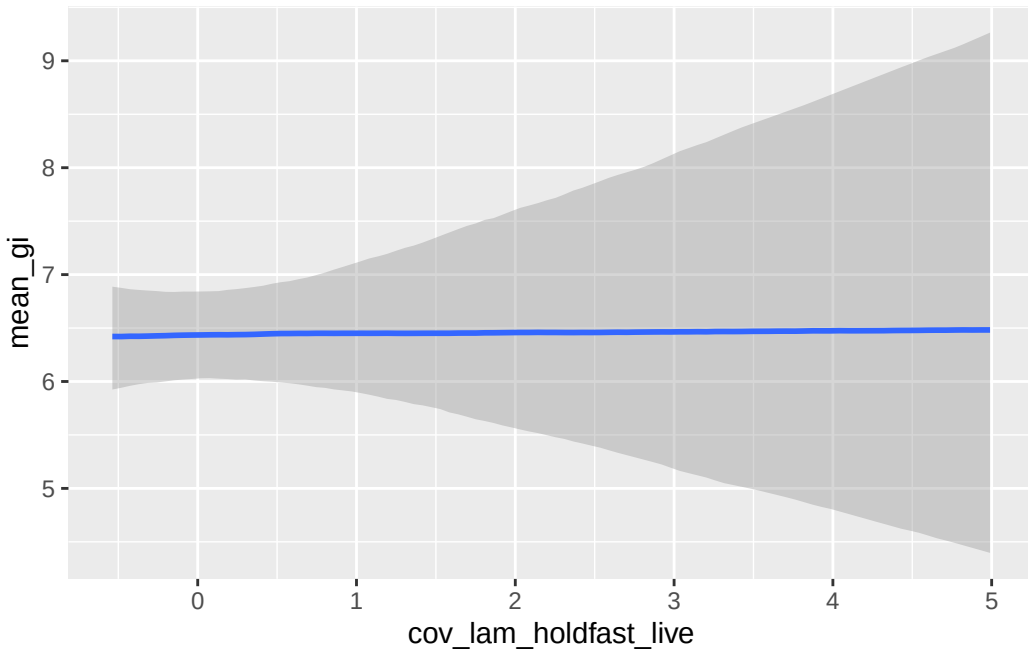

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



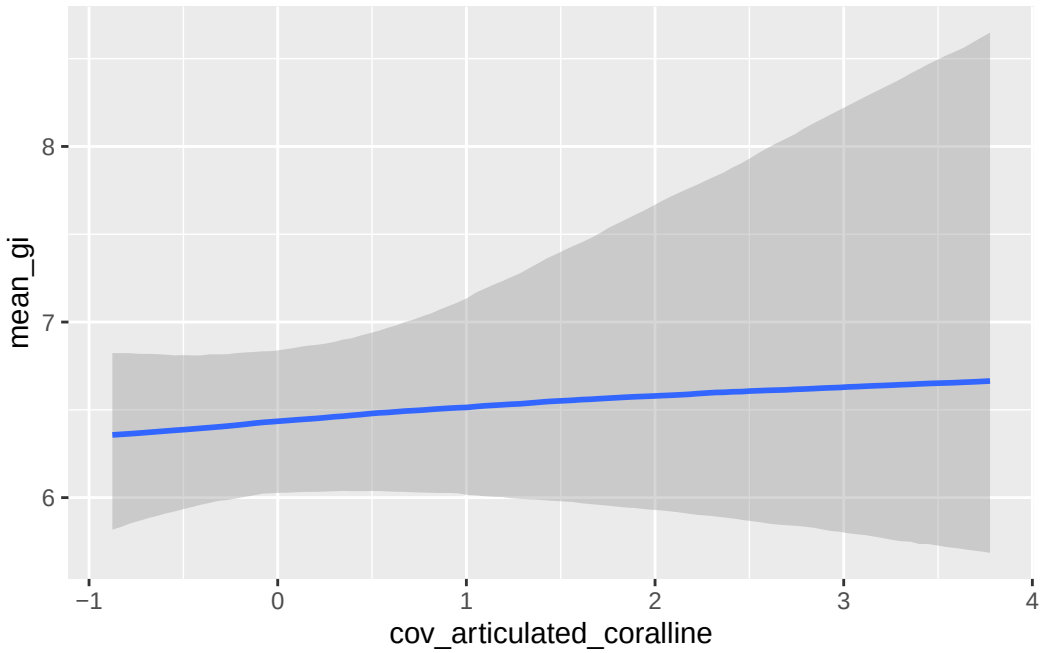

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



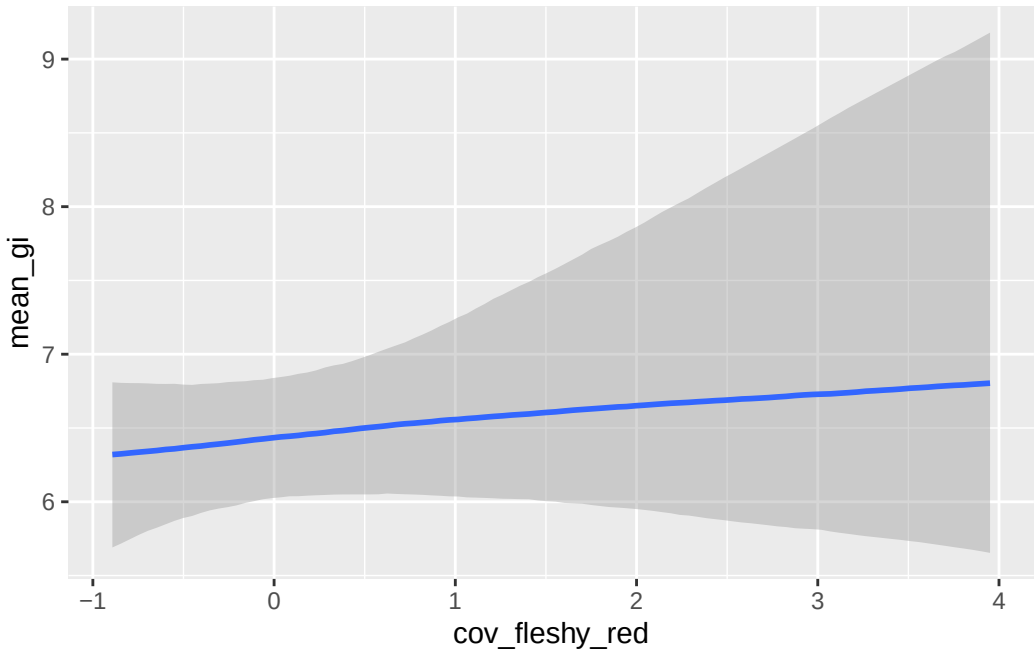

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



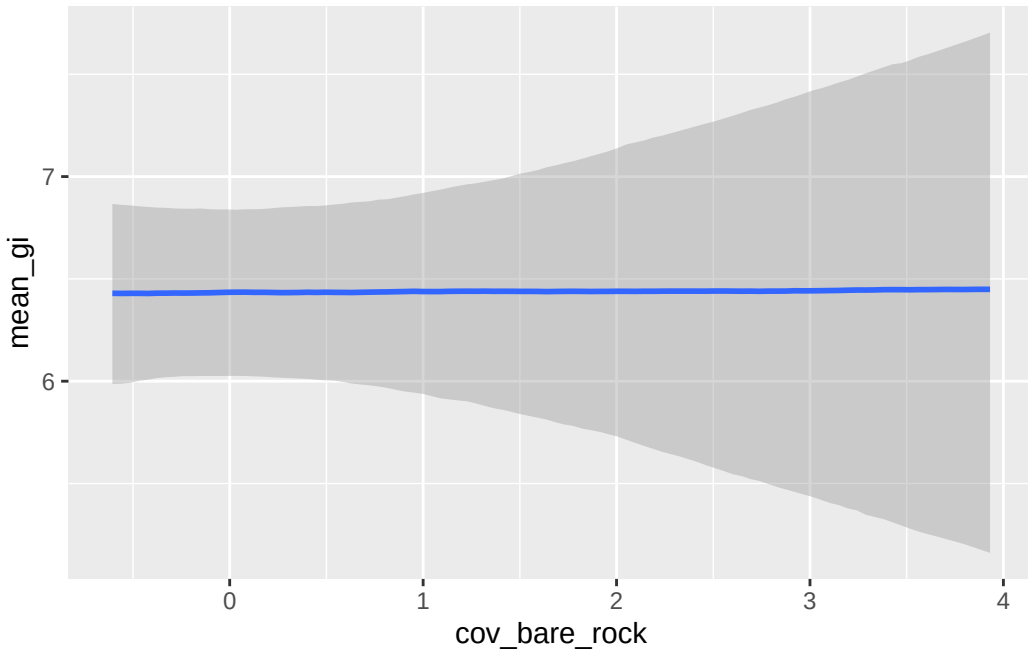

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



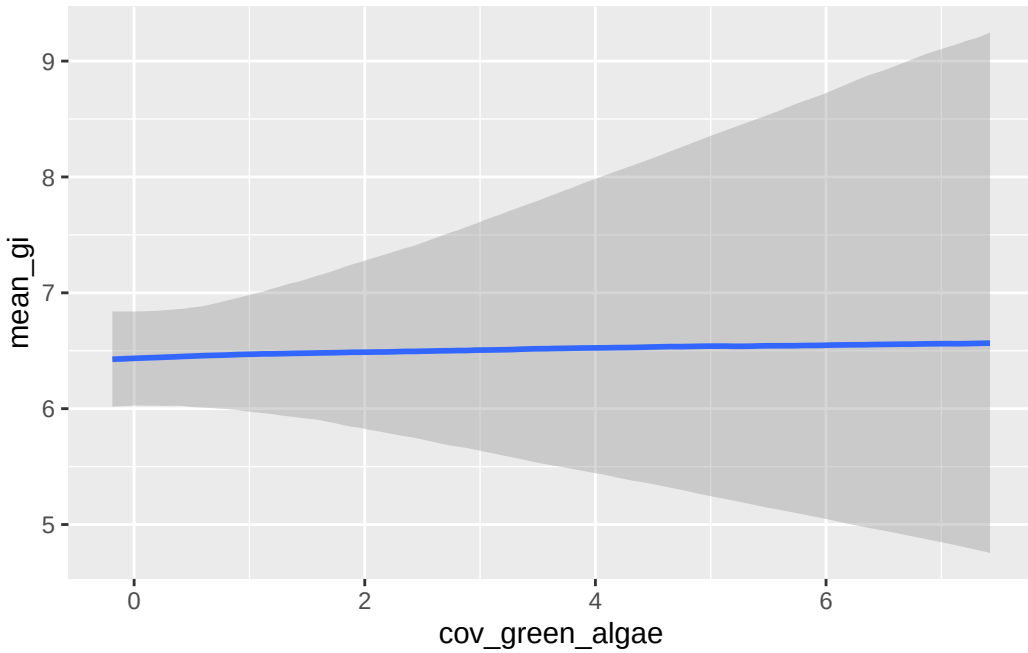

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



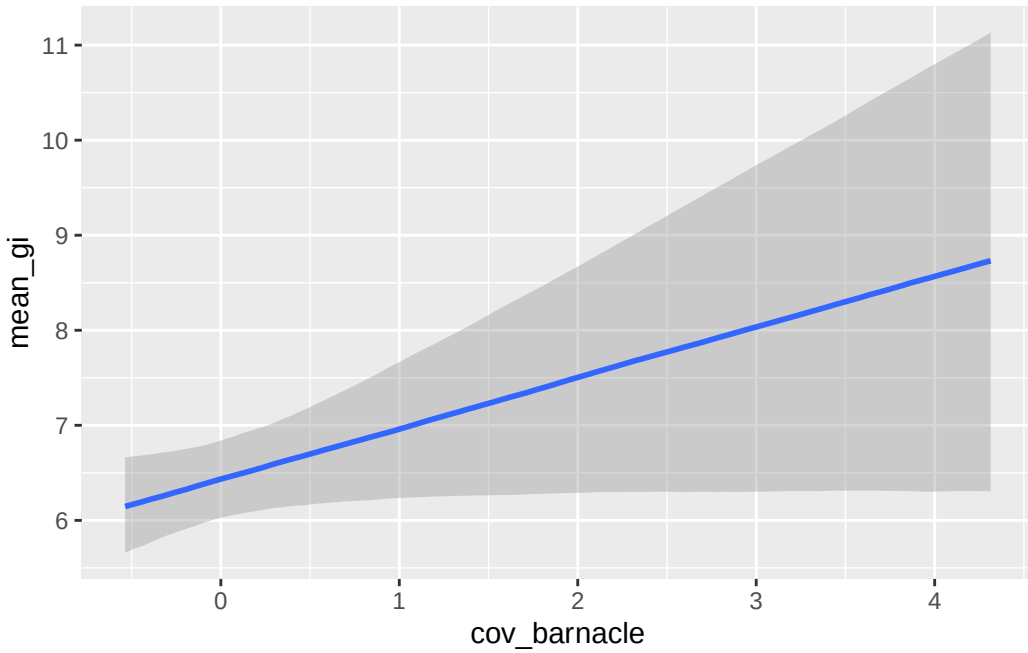

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



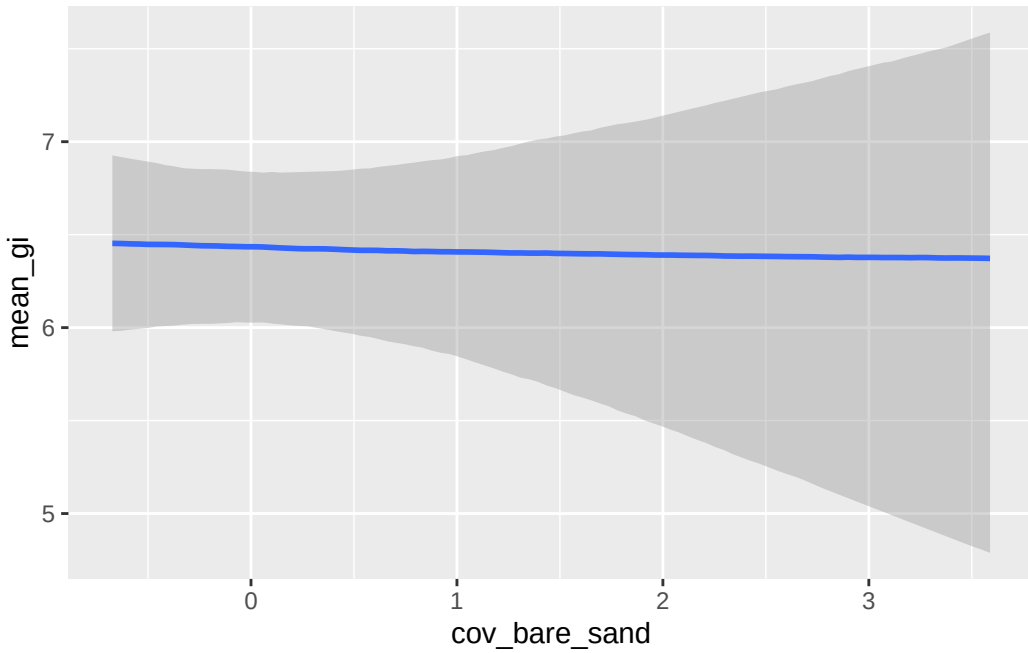

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



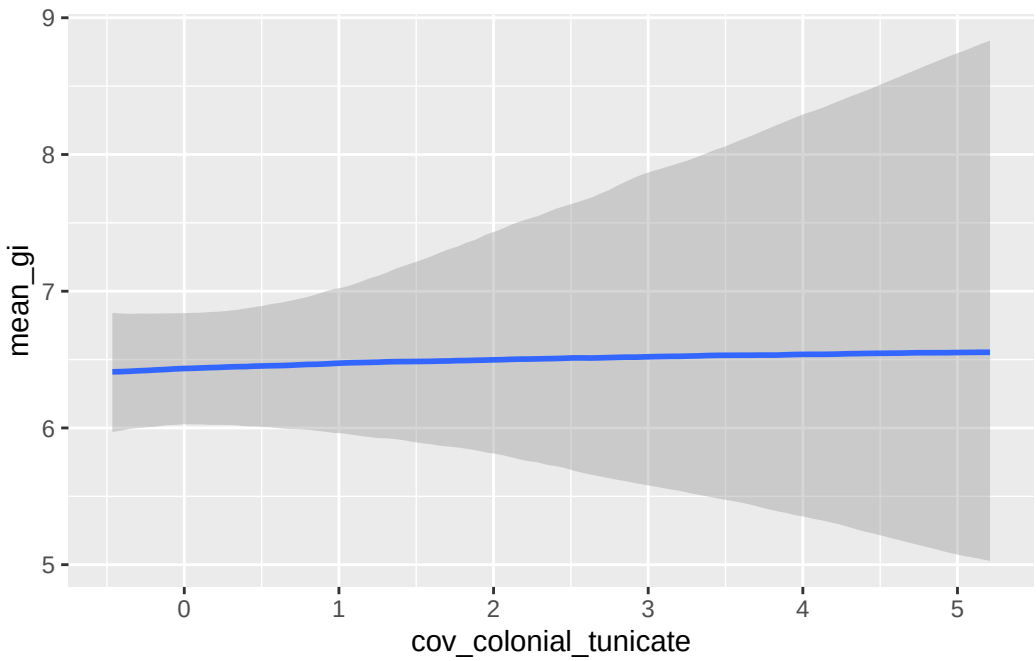

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



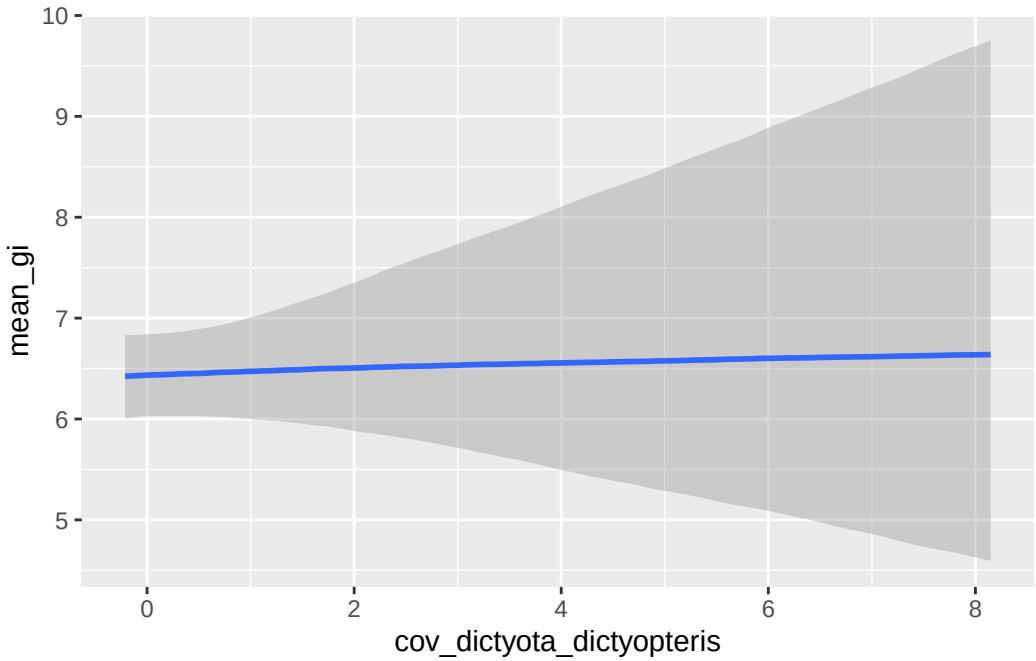

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



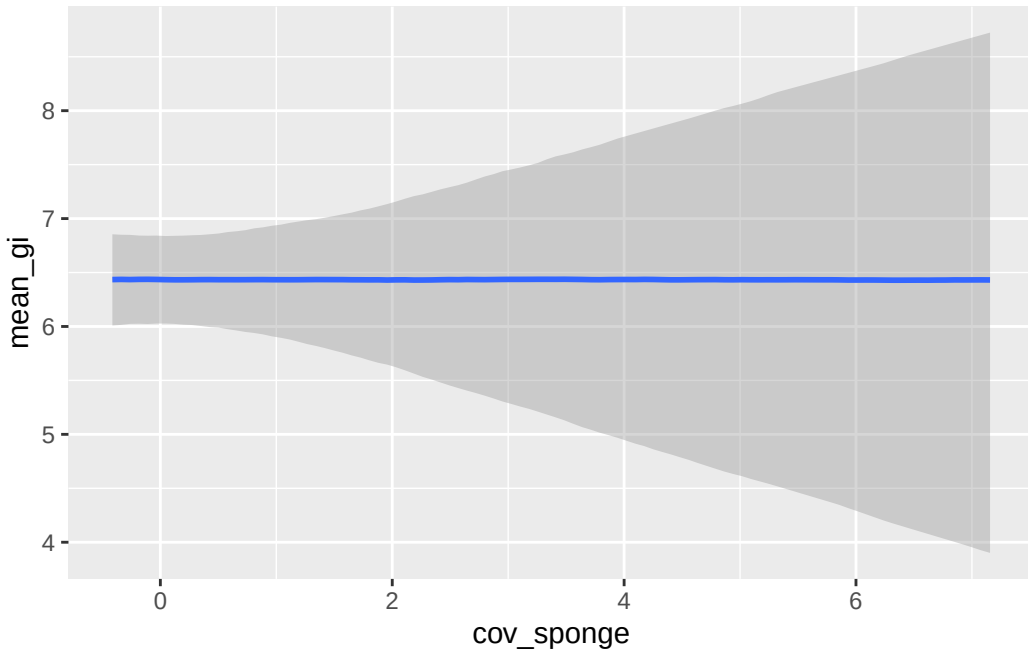

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



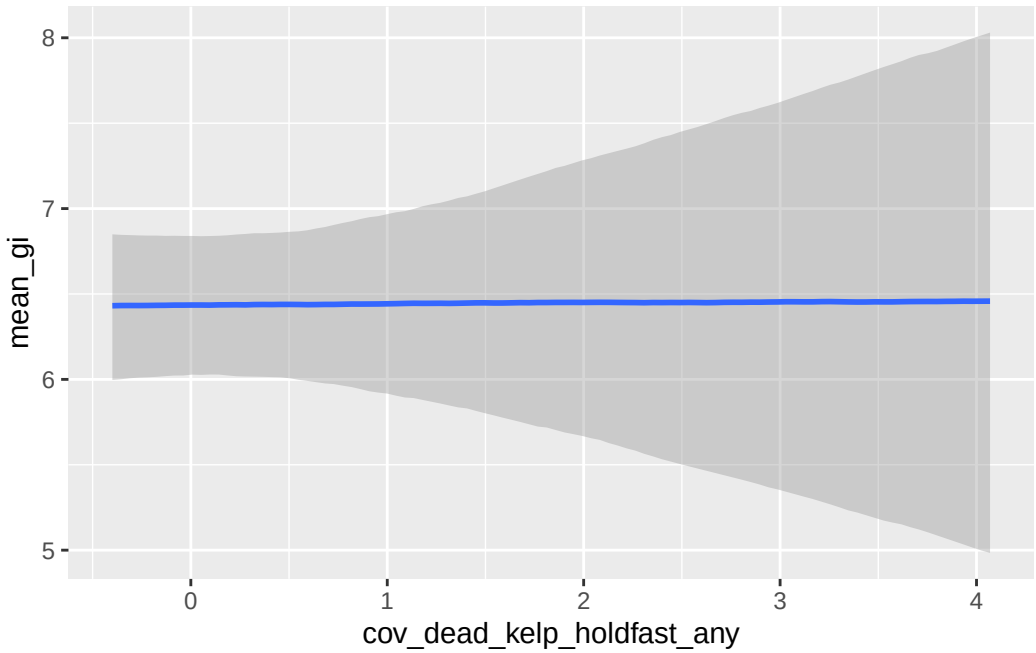

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



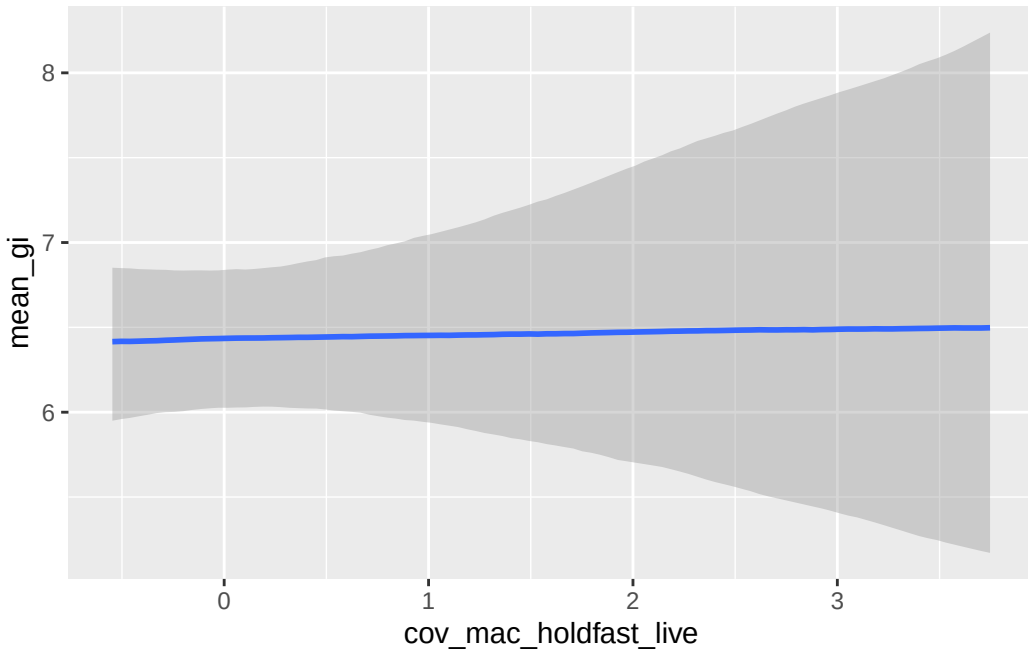

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



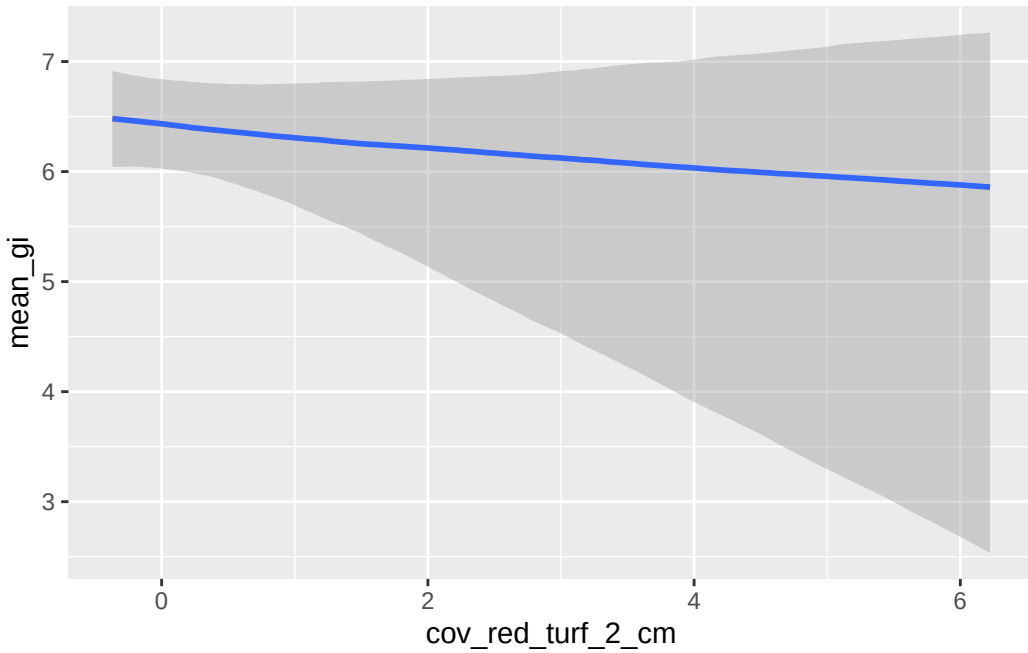

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



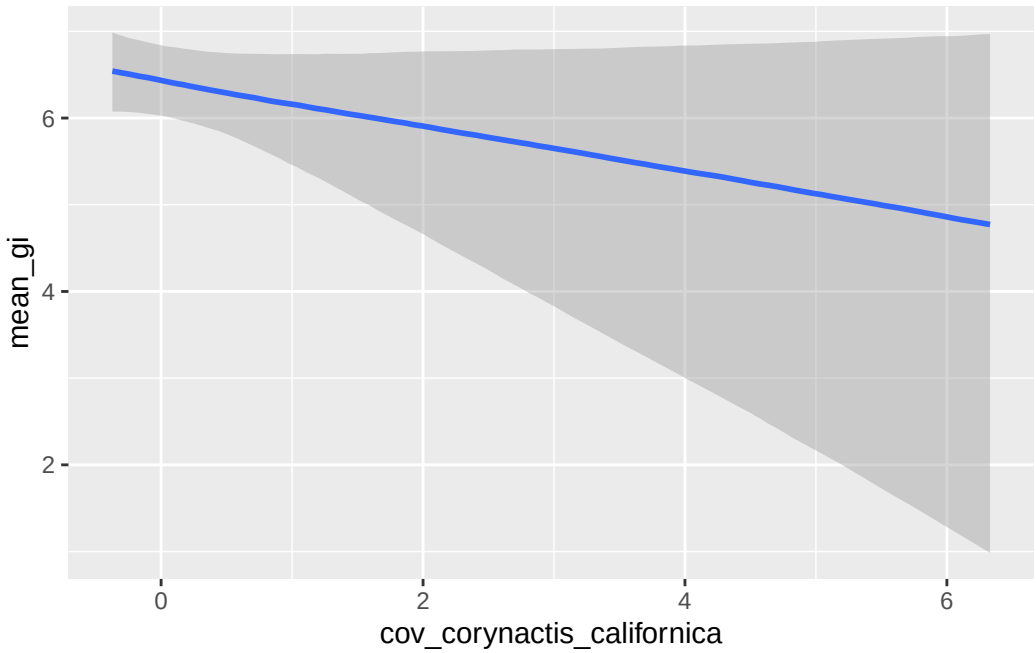

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



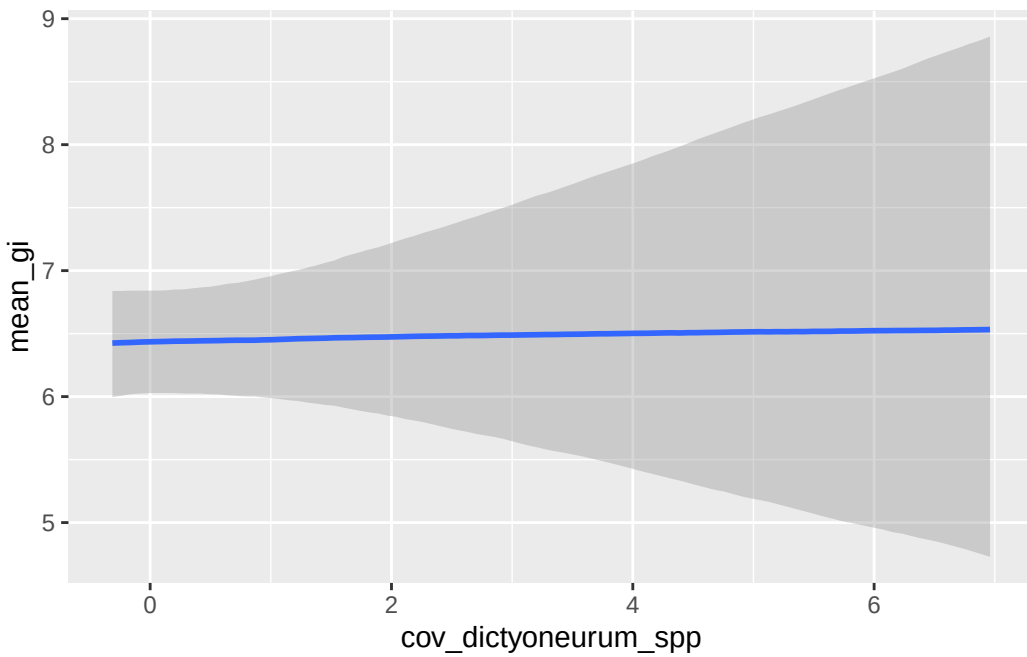

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```




```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```



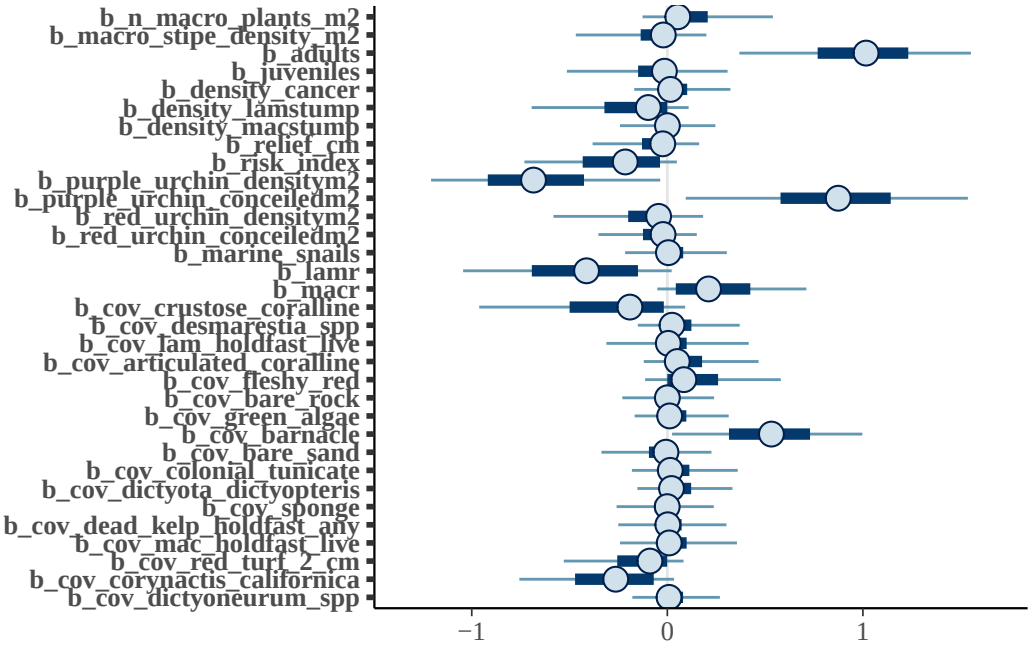

```
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
Ignoring unknown labels:
* fill : "NA"
* colour : "NA"
```




```r
bayes_R2(brm_hs, pars = significant_vars)
```

```
    Estimate  Est.Error      Q2.5     Q97.5
R2 0.5623551 0.06436701 0.4161156 0.6662897
```

```r
library(bayesplot)
posterior <- as_draws_df(brm_hs)
coef_cols <- grep("^b_", colnames(posterior), value = TRUE)
coef_cols <- coef_cols[coef_cols != "b_Intercept"]  # remove intercept
mcmc_intervals(posterior, pars = coef_cols)
```




```r
coef_summary <- posterior %>%
  summarise(across(all_of(coef_cols), ~ median(.))) %>%
  pivot_longer(everything(), names_to = "predictor", values_to = "median")
top_predictors <- coef_summary %>%
  filter(abs(median) > 0.05) %>%  # threshold can be adjusted
  pull(predictor) %>%
  gsub("^b_", "", .)  # remove b_ prefix to match PCA loadings
loadings <- as.data.frame(pca$rotation)
top_loadings <- loadings[top_predictors, 1:4]  # first 4 PCs
top_loadings$predictor <- rownames(top_loadings)
top_loadings
```

```
                                PC1         PC2         PC3         PC4
n_macro_plants_m2        0.33732141  0.21563058  0.14368976  0.07629640
NA                               NA          NA          NA          NA
density_lamstump         0.12713800 -0.16334794 -0.44607187 -0.10897324
NA.1                             NA          NA          NA          NA
purple_urchin_densitym2 -0.16674531  0.08129745 -0.43963162 -0.14874706
purple_urchin_conceiledm2  0.27615115 -0.27839266 -0.32593116 -0.10565139
NA.2                             NA          NA          NA          NA
macro_stipe_density_m2   0.35741664 -0.05645212 -0.18095603 -0.01870642
cov_crustose_coralline  -0.30822437  0.15413830 -0.19345002 -0.17882835
```

```
cov_articulated_coralline  -0.02160984 -0.36977641  0.20138136  0.28799480
NA.3                                NA          NA          NA          NA
cov_barnacle                0.20088044  0.29185535 -0.02008689  0.28877869
NA.4                                NA          NA          NA          NA
cov_corynactis_californica -0.10302370  0.37316137 -0.10359940  0.19987886
                                            predictor
n_macro_plants_m2                   n_macro_plants_m2
NA                                                 NA
density_lamstump                     density_lamstump
NA.1                                             NA.1
purple_urchin_densitym2       purple_urchin_densitym2
purple_urchin_conceiledm2   purple_urchin_conceiledm2
NA.2                                             NA.2
macro_stipe_density_m2         macro_stipe_density_m2
cov_crustose_coralline         cov_crustose_coralline
cov_articulated_coralline   cov_articulated_coralline
NA.3                                             NA.3
cov_barnacle                               cov_barnacle
NA.4                                             NA.4
cov_corynactis_californica cov_corynactis_californica
```

```r
long_loadings <- top_loadings %>%
  pivot_longer(cols = starts_with("PC"), names_to = "PC", values_to = "loading")
```